GENERATED DOCUMENT

Product Requirements Document

Newsletter Builder

Version 0.1.0 · commit 9abc092

November 6, 2026

Generated from source code

Reverse-engineered, **read-only**, from source code, the automated test suite, `docs/PROGRESS.md`, and `docs/brd/` — no source file was changed to produce this document. Every requirement cites a BRD business-rule id (`BR-xx`) where one applies, or states "no BRD link." Every non-obvious statement is tagged **Fact** (default), **Inference:** (a conclusion drawn from behavior, not stated verbatim), or **Not Found in Codebase** (looked for, confirmed absent). See `VERIFICATION_NOTES.md` for the complete list of every Inference/Not-Found item.

Verified at write time (not quoted from memory): `npx tsc --noEmit` → 0 errors. `npm test` → 41 test files, 950 tests, all passing.

ID conventions

Prefix	Meaning	Defined in	Stable across parts?
`F-xxx`	A feature from the inventory	Part 1 §4	Yes — same id used in Parts 2–4
`PR-xxx`	A functional requirement	Part 3 §8	Yes — same id used in Part 4 §15
`BR-xx`	A business rule from the BRD	`docs/brd/BRD-02.md` §13	Cited, never redefined, here

No id is renumbered between parts. "No BRD link" means the BRD has no matching business rule for that item — it is a real, shipped capability, just not one the BRD formalized as a numbered rule.

Table of Contents

Part 1 — PRD-01-overview.md

1. Product Overview
2. Goals, Non-Goals, and Success Metrics
3. Personas and Use Cases
4. Feature Inventory

Part 2 — PRD-02-feature-specs.md

5. Detailed Feature Specifications — one spec per feature, F-001 through F-023

Part 3 — PRD-03-flows-and-requirements.md

6. User Flows
7. Screen and Panel Inventory
8. Functional Requirements (PR ids)

9. Product-Level Non-Functional Requirements
10. Data From the User's Perspective

Part 4 — PRD-04-release-and-appendix.md

11. Release Scope
12. Risks and Open Questions
13. Analytics and Telemetry
14. QA and Acceptance — Manual Verification Checklist
15. Traceability Matrix
16. Glossary

Verification Notes

Every Inference and Not-Found-in-Codebase item, listed by section, with reasoning.

One-paragraph orientation

Newsletter Builder is a single-user, browser-only drag-and-drop email builder with no backend, no login, and no analytics. This PRD inventories 23 shipped features (`F-001`–`F-023`), 71 functional requirements (`PR-001`–`PR-071`, MoSCoW-prioritized as Inference), and cross-references every one against the 30 business rules already documented in `docs/brd/`. Where a typical PRD template expects a vision statement, a prioritization document, analytics, or a release process, this one says "Not Found in Codebase" rather than inventing one — the product genuinely does not have those artifacts today.

Part 1 of 4 — Sections 1–4: Overview, Goals, Personas, Feature Inventory

> **Verified-now facts used throughout this PRD** (run at the time of writing, not quoted from memory or from `docs/PROGRESS.md`'s own log): `npx tsc --noEmit` — **0 errors**. `npm test` (vitest) — **41 test files, 950 tests, all passing**, 70.46s. These numbers match `docs/PROGRESS.md`'s final entry exactly, confirming this PRD describes the same codebase state as the BRD.

---

1. Product Overview

What it is

Newsletter Builder is a **browser-only, single-user, drag-and-drop email template builder** (React 19 + TypeScript + Vite + Tailwind CSS). A marketing/communications user assembles an HTML newsletter visually and exports email-client-safe HTML plus a plain-text alternative. (Fact — `README.md` lines 1–5.)

Vision

Not Found in Codebase as a written vision statement (no `VISION.md`, no product-strategy doc, no mission statement anywhere in the repo). **Inference**, synthesized from the product's own framing across `README.md` and `docs/PROGRESS.md`: *"Let a marketing/communications team produce polished, on-brand, email-client-safe newsletters without a developer, a designer, or hand-written HTML in the loop — and, increasingly, without writing the first draft of the copy or sourcing the first image by hand either."*

The problem

Problem (as framed in the product's own history)	Evidence
Hand-coded HTML email is fragile across clients (Outlook/Gmail/Apple Mail render differently)	Fact — the entire renderer (`src/lib/render.ts`) exists specifically to emit table-based, inline-styled, Outlook-safe markup
Re-theming an email for a different brand meant rebuilding it	Fact — `docs/PROGRESS.md` Feature 1A
Common sections (header, hero, article list, footer) were rebuilt by hand every time	Fact — Feature 1B
No visibility into whether an email would actually work in an inbox before sending	Fact — Feature 3A ("one engine that reads what is actually sent and says what an inbox will do with it")
Starting a draft from scratch is slow	Fact — Feature 6A's own framing
On-brand imagery required a separate tool	Fact — Feature 5A

Positioning

Inference (no competitive-positioning document exists): a lightweight, **internal, no-backend** alternative to a hosted ESP's own drag-and-drop editor (e.g., Mailchimp's/HubSpot's builder) or a general design tool (Canva/Figma) repurposed for email — differentiated specifically by (a) running entirely client-side with zero account/hosting cost, (b) a built-in, non-blocking email-quality engine, and (c) bring-your-own-key AI assistance for both copy and imagery, inline in the same editor.

What the product is NOT

Not	Evidence
Not an ESP / sending tool	Fact — `README.md`: "No sending: export the HTML and send it through your ESP (SES, SendGrid, ...)"
Not a multi-user / collaborative tool	Fact — everything is per-browser `localStorage` / `IndexedDB`; no sync of any kind (§10)
Not an accounts/auth product	Fact — `src/lib/user.ts`'s own header comment: "no login, no roles, an internal tool's substitute for an account"
Not an image-hosting service	Fact — `imageHosting.configured === false`; AI-generated images always export as a placeholder until manually rehosted elsewhere
Not a test-send or inbox-preview service (e.g., Litmus/Email on Acid)	Fact — README: "Always test final HTML in the clients you care about (Litmus / Email on Acid / real inboxes)"
Not a managed/metered AI product	Fact — the user's own OpenAI key is billed directly by OpenAI; the app has no cost visibility or usage caps (confirmed by exhaustive grep across `src/lib/ai/*`, no cost/quota/pricing code found)
Not a CMS or content-approval workflow tool	Fact — exactly one functional role exists (§3); the only "approval" is a single non-blocking Checks warning (`AI_DRAFT_UNREVIEWED`)
Not a general page-builder (web pages, landing pages)	Inference — every renderer constraint (table layout, no flex/grid, inline styles) is email-specific; nothing in the product targets a browser-rendered destination

---

2. Goals, Non-Goals, and Success Metrics

Goals (Inference — derived from the pattern of shipped features; no OKR/goals document exists)

#	Goal
G1	Let a non-developer produce email-client-safe HTML without hand-coding
G2	Keep every email visually on-brand with minimal per-email effort
G3	Catch inbox-breaking mistakes before a human sends, without ever blocking the export
G4	Reduce first-draft effort via AI assistance for copy and imagery, always behind a human-visible review signal
G5	Protect against data loss (auto-save, version history, undo/redo)
G6	Make content reuse cheap (saved blocks, image library, presets)
G7	Keep AI usage safe and auditable (keys never serialized; redacted errors; no silent double-billing retries)

Non-goals (Fact, each explicitly evidenced — not merely unplanned)

#	Non-goal	Evidence
NG1	Sending email	README "No sending"
NG2	Hosting images	`imageHosting.configured: false` (`image-storage.ts`)
NG3	Multi-user collaboration / cross-device sync	No backend; `localStorage`/`IndexedDB` only
NG4	Authentication / roles / permissions	`user.ts` header comment; `canEditBrand()`/`getAccessibleBrands()` always return "true"/"everything"
NG5	AI usage metering or cost control	No cost/quota code found anywhere in `src/lib/ai/*`
NG6	Link-liveness checking	`checks.ts` comment: "Links are classified, not fetched"
NG7	Localizing the *application* UI (as opposed to AI-generated *content*, which does support 3 languages)	No i18n framework in `src/`; all UI strings are hardcoded English JSX

Success metrics

Not Found in Codebase: any analytics, telemetry, or usage-metrics instrumentation of any kind. Exhaustive — no event tracking, no error-reporting SDK, no aggregation, no dashboard (confirmed again for this PRD; see §13). There is therefore **no metric that is "measured" by the product over time or across sessions.**

What follows instead is the closest honest substitute: **in-the-moment, per-document counters the UI itself displays**, which a user could read off the screen as a manual proxy — these are not KPIs, have no history, and are never aggregated.

Observable in-app counter	Where shown	What it is a proxy for
Checks severity counts (errors/warnings/info)	Top-bar badge, Checks drawer, Export dialog	"Is this specific email currently in-box-safe?"
Export size in KB	Export dialog	"Will Gmail clip this specific email?" (soft cap 90KB, hard cap 102KB)
UTM "tagged N, skipped M"	Export dialog	"Did link tracking apply where expected, for this specific email?"
Storage usage (bytes of ~5MB budget)	Templates modal, Settings → Data & storage	"How close is this browser to running out of local storage?"
Test-suite pass count	`npm test` (developer-facing, not in-product)	Engineering health, not a product/business metric

---

3. Personas and Use Cases

This is confirmed, per `docs/PROGRESS.md` Feature 1A and `src/lib/user.ts`, an **internal tool with exactly one functional role and no authentication.** The persona table below documents what actually exists, not a hypothetical multi-role model.

Persona	Who (Fact)	Goals	What they can do	What they cannot do
Marketing/Communications editor	The one role that exists — `docs/PROGRESS.md` names the user base "the Vee MARCOM team"	Produce an on-brand, inbox-safe newsletter quickly	Everything — every screen, every action, with no restriction	Nothing is restricted; there is no second role to compare against

Not Found in Codebase: a brand-owner-only role, an approver role, a viewer-only role, or an admin role — `canEditBrand()` / `getAccessibleBrands()` exist as **named, in-code future seams** (1A) that currently grant full access unconditionally.

Use cases (what the one persona actually does, each traceable to a feature in §4)

Use case	Primary feature(s)
Build a newsletter from a blank canvas	F-001, F-002, F-003, F-004
Start from a ready-made layout	F-005
Re-theme an email for a different brand	F-006
Make the email look correct in dark-mode inboxes	F-007
Personalize an email per-recipient without a second file per ESP	F-008
Track link clicks via campaign parameters	F-009
Produce a text-only fallback for the HTML	F-010
Produce the final deliverable to paste into an ESP	F-011
Catch inbox-breaking problems before sending	F-012
Reuse a block/row built once across many emails	F-013
Not lose work when a tab closes unexpectedly	F-014
Recover an earlier version of an email	F-015
Maintain a vetted library of image URLs	F-016
Generate an on-brand image without leaving the editor	F-017
Turn a short brief into a first-draft email	F-018

Use case	Primary feature(s)
Choose which AI model handles which task	F-019
Keep AI working if one model/provider is briefly unavailable	F-020
Connect and manage an OpenAI API key	F-021
Manage profile, defaults, and local data from one place	F-022
Move brands/templates/blocks to another browser, or back up/wipe local data	F-023

4. Feature Inventory

Status legend: Complete = fully implemented, tested, documented as shipped in `docs/PROGRESS.md`. Partial = implemented but with a stated, deliberate gap (e.g., no hosting). Not built = referenced as a future idea only, nothing implemented.

ID	Feature	Area	Description	Status	Screens	Source files (key)	BR link
F-001	Canvas & editing core	Editor	Selection model, drag-and-drop (dnd-kit), undo/redo (100-step history with edit-coalescing), keyboard shortcuts (Ctrl+Z/Shift+Z/Y, Ctrl+D, Delete, Escape); the same `renderBlock()` drives canvas and export	Complete	Canvas, TopBar	`src/components/Canvas.tsx`, `src/components/dnd.tsx`, `src/store.ts`, `src/App.tsx`	no BRD link

ID	Feature	Area	Description	Status	Screens	Source files (key)	BR link
F-002	Row layouts & columns	Editor	1–4 column rows with ratio layouts (e.g. 1:2, 2:1, 1:2:1), row/content background, padding, stack-on-mobile toggle; resizing a row merges overflow blocks into the last kept column (never deletes)	Complete	LeftPanel (Rows tab), RightPanel (Row editor)	`src/lib/design-ops.ts: apply-RowLayout`, `src/lib/defaults.ts: ROW_LAYOUTS`	no BRD link
F-003	Atomic content blocks	Editor	9 block types: Heading (H1–H6), Text (rich text via TipTap), Image, Button, Divider, Menu, Social, Video (thumbnail only), HTML (raw escape hatch)	Complete	LeftPanel (Content tab), RightPanel editors	`src/lib/defaults.ts: BLOCK_META`, `src/components/editors.tsx`, `src/lib/render.ts`	BR-18 (Article-list cap, related repeater discipline)

ID	Feature	Area	Description	Status	Screens	Source files (key)	BR link
F-004	Composite blocks	Editor	4 composite types built by reusing atomic renderers internally: Header (logo+nav), Hero (5 layouts incl. 2/3-card variants), Footer (brand-sourced address/legal/unsubscribe/social), Article list (3 layouts, ≤10 items)	Complete	RightPanel editors (`HeaderEditor`, `HeroEditor`, `FooterEditor`, `ArticleListEditor`)	`src/lib/defaults.ts`, `src/lib/render.ts` (composite cases)	BR-18
F-005	Starter presets	Editor / Onboarding	8 brand-neutral presets (Newsletter, Promotional, Product launch, Welcome, Cold outreach, Event invitation, Order confirmation, Weekly digest) + Blank, each with a live real-HTML thumbnail	Complete	NewEmail Modal (Presets tab)	`src/lib/presets.ts`, `src/components/NewEmailModal.tsx`	BR-24, BR-25

ID	Feature	Area	Description	Status	Screens	Source files (key)	BR link
F-006	Brand kit & brand switcher	Branding	Named brand kits (colors, fonts, button style, logo pairs, footer, social links, voice guidance); color/font fields are tokens resolved at render time; switching brands re-themes without touching content	Complete	BrandsModal, TopBar (switcher), NewEmailModal (brand picker)	`src/lib/brand.ts`, `src/components/BrandsModal.tsx`, `src/store.ts: setBrandId`	BR-01, BR-02, BR-03, BR-04
F-007	Dark mode support	Branding / Export	Per-brand dark palette (or derived); additive-only export (byte-identical with the toggle off); token-vs-literal color rule; dark-specific contrast checks	Complete	BrandsModal (Dark mode section), TopBar/PreviewModal (scheme toggle)	`src/lib/dark.ts`, `src/lib/output.ts: darkRenderOpts`	BR-22, BR-23
F-008	Merge tags	Export / Personalization	Canonical `{{tag}}` storage; 6 built-ins; 4 output formats (curly/Mailchimp/HubSpot/custom) applied only at export	Complete	RightPanel (Body/Email settings), Export dialog	`src/lib/merge.ts`	BR-21

ID	Feature	Area	Description	Status	Screens	Source files (key)	BR link
F-009	UTM link tagging	Export / Tracking	Off by default; applies at export only; preserves existing query/hash; never double-tags; per-link opt-out; unsubscribe/view-in-browser links never tagged but still counted	Complete	RightPanel (Body/Email settings), Export dialog	`src/lib/utm.ts`	BR-19, BR-20
F-010	Plain-text export	Export	Generated from the already-resolved design (so UTM/merge formatting match); 72-char wrap; URLs never split	Complete	Export dialog (Plain text tab)	`src/lib/plaintext.ts`	no BRD link (governed by the same boundary as F-008/F-009)

ID	Feature	Area	Description	Status	Screens	Source files (key)	BR link
F-011	HTML export & live preview	Export	`resolveForOutput()` pipeline (brand → AI-draft placeholder swap → UTM → merge format) feeds one renderer for both the live canvas and the exported HTML; Preview modal shows the exact exported document in a sandboxed iframe	Complete	ExportHtmlModal, PreviewModal	`src/lib/output.ts`, `src/lib/render.ts`, `src/components/Modals.tsx`	BR-01, BR-05
F-012	Checks panel (quality engine)	Quality	Non-blocking engine covering size, links, unsubscribe, subject/preheader, WCAG AA contrast (light+dark), text/tap-target size, thin content, every image rule, AI-draft rules; stable per-issue ids enable persistent per-email "Ignore"	Complete	ChecksPanel (drawer), TopBar badge, Export dialog summary	`src/lib/checks.ts`, `src/lib/contrast.ts`, `src/components/ChecksPanel.tsx`	BR-05, BR-07

ID	Feature	Area	Description	Status	Screens	Source files (key)	BR link
F-013	Saved blocks library	Reuse	Freeze a selected block/row into a named, categorized, searchable library; brand tokens stay tokens (re-theme on reuse); insert always mints fresh ids	Complete	LeftPanel (Saved tab), SaveBlockDialog	`src/lib/blocks-library.ts`, `src/components/SaveBlockDialog.tsx`	BR-25
F-014	Auto-save	Data / Reliability	2-second debounce; creates the record on first edit; `beforeunload` flush; status always visible, including "Not saved: storage full"	Complete	TopBar (save status), Templates modal	`src/store.ts: saveNow`, `src/lib/storage.ts: writeSafe`	BR-11, BR-13
F-015	Version history	Data / Reliability	≤20 snapshots per email; manual/auto(≤1/5min)/pre-restore/pre-replace reasons; dedup on identical design; restore snapshots the present first	Complete	HistoryModal	`src/lib/versions.ts`, `src/components/HistoryModal.tsx`	BR-12, BR-13

ID	Feature	Area	Description	Status	Screens	Source files (key)	BR link
F-016	Image library (URL-based)	Assets	Vetted https-only image URLs with name/default-alt/tags/brand scope; picker on 12 image-bearing fields app-wide; on-demand load probing; file upload deliberately refused (no hosting)	Partial (URL-only by design; no upload/hosting)	LeftPanel (Images tab), ImagePickerModal	`src/lib/image-storage.ts`, `src/lib/image-audit.ts`, `src/components/ImageLibrary.tsx`	BR-17
F-017	AI image generation	AI	Bring-your-own-key OpenAI image generation; one request per generation (no auto-retry); magic-byte response validation; result is a local IndexedDB "draft," never hosted; export ships a placeholder until the user manually rehosts	Partial (generation complete; hosting explicitly out of scope)	AiImagePanel	`src/lib/ai/openai.ts`, `src/lib/ai/validate.ts`, `src/lib/drafts.ts`, `src/components/AiImagePanel.tsx`	BR-06, BR-08

ID	Feature	Area	Description	Status	Screens	Source files (key)	BR link
F-018	Generate email from a brief (AI)	AI	Structured brief (topic/audience/goal/key points/tone/length/CTA/language/source text) → one AI text-generation call against a strict JSON schema, derived automatically from the chosen preset's own block tree → mandatory human review screen before anything is written to the design	Complete	NewEmailModal (Brief tab), BriefForm, BriefReview	`src/lib/ai/create-from-brief.ts`, `src/lib/ai/slots.ts`, `src/lib/ai/brief-validate.ts`	BR-26, BR-27, BR-28
F-019	Per-task AI model routing	AI / Settings	Independent provider/model/option choice for `text.brief` vs. `image.generate`; a per-generation Quality selector always wins over the task's configured default	Complete	Settings → AI ("Models per task")	`src/lib/ai/tasks.ts`, `src/lib/ai/task-settings.ts`, `src/lib/ai/run-task.ts`	BR-29 (fallback logic shared with F-020)

ID	Feature	Area	Description	Status	Screens	Source files (key)	BR link
F-020	Provider/model fallback chains	AI / Settings	Ordered, user-configured fallback list per task, tried only on network/timeout/rate-limit failures; a model-access denial ("No access to `<model>`") is a session-only, per-task signal	Complete	Settings → AI (fallback editor, status chip)	`src/lib/ai/run-task.ts`, `src/lib/ai/model-access.ts`	BR-29
F-021	AI provider & API key management	AI / Settings	One key per provider; default in-memory-only, opt-in "Remember on this device" (disclosed as unencrypted); key only persisted after a passing "Test and save"; never enters the app's state store or backups	Complete	AiImagePanel (gear icon) / Settings → AI (shared `SettingsView` component)	`src/lib/ai/key-store.ts`, `src/lib/ai/provider.ts: redactKeys`	BR-08, BR-30

ID	Feature	Area	Description	Status	Screens	Source files (key)	BR link
F-022	Settings hub & profile	Settings	Two-level navigation hub (Profile, AI, Brands, Email defaults, Checks, Data & storage, About); a local, unauthenticated display-name "profile" used only for `createdBy` / `updatedBy` attribution	Complete	SettingsHub + 7 section components, ProfileBar	`src/components/settings/SettingsHub.tsx`, `src/lib/user.ts`	BR-10

ID	Feature	Area	Description	Status	Screens	Source files (key)	BR link
F-023	Import / export & backup	Data gover-nance	Per-email JSON export/import; brand JSON export/import; saved-block JSON export/import; image-library JSON export/import; one-file full backup/restore (brands+templates+saved blocks+images+versions, deliber-ately ex-cluding AI keys/drafts); "Clear all local data" gated by typing `DELETE`	Complete	TopBar (JSON import/export), BrandsModal, LeftPanel (Saved/Images tabs), Settings → Data & storage	`src/lib/backup.ts`, `src/lib/clear-all.ts`, `src/lib/design-ops.ts: normal-izeDesign`	BR-09, BR-15, BR-16

Feature count: 23 (F-001–F-023). **Status breakdown:** 21 Complete, 2 Partial (F-016 image library — URL-only by design; F-017 AI image generation — no hosting by design). **0 Not built** — every feature named in the task's required coverage list exists in the shipped product; "Not built" items (e.g., test-email sending, a second AI provider) are documented as backlog candidates in Part 4 §11, not as inventory rows, since they have no code to inventory.

Part 2 of 4 — Section 5: Detailed Feature Specifications

> One spec per feature from the inventory in `PRD-01-overview.md` §4. Same `F-xxx` ids, not renumbered. Acceptance criteria are **Inference** (phrased from tested behavior; no formal Given/When/Then document exists in the repo) unless noted otherwise.

F-001 — Canvas & editing core

User story: As the editor, I want to add, arrange, and edit blocks on a live preview canvas, so that what I see is exactly what will be exported. **Trigger:** Any click/drag on the canvas or left-panel tiles; any keyboard shortcut. **Flow:** Click/drag a tile from LeftPanel → `addBlock`/`insertBlock` (store action) → `commit()` pushes an undo step → canvas re-renders via the same `renderBlock()` used by export. **UI behaviour:** Selecting a block/row shows its editor in RightPanel; a hidden Footer (showFooter off) still renders on canvas at reduced opacity with a "Hidden on send" badge, still selectable/deletable. **Rules:** `commit()` is the single mutation gateway; rapid same-field edits (slider drag, typing) coalesce into one undo step within a 900ms window; undo/redo clears a stale selection if the selected block/row no longer exists; history capped at 100 steps (in-memory, not persisted). **Edge cases:** Deleting the last block in the last row leaves an empty canvas (not a crash); undo past the start or redo past the end is a no-op (`canUndo`/`canRedo` guard the toolbar buttons). **Error states:** None surfaced to the user — structural edits cannot fail (no network/storage call in the mutation path itself; persistence failures are handled separately by F-014). **Dependencies:** F-002 (rows), F-003/F-004 (blocks), F-006 (brand resolution for live preview). **Acceptance criteria:**

```
Given a block is selected
When Ctrl+D is pressed
Then a duplicate is inserted immediately after it and selected

Given 5 rapid edits to the same text field within under 900ms of each other
When Ctrl+Z is pressed once
Then all 5 edits are undone together, not one at a time
```

F-002 — Row layouts & columns

User story: As the editor, I want multi-column rows with different ratios, so that I can lay out content side by side. **Trigger:** Picking a layout tile in LeftPanel's Rows tab, or changing layout on an existing row in RightPanel. **Flow:** `addRow(layout)` or `setRowLayout(rowId, layout)` → `applyRowLayout()` resizes the `columns` array. **UI behaviour:** A visual ratio preview (proportional bars) on each layout tile; the row's content/background color fields distinguish "row background" (can bleed full width) from "content background" (confined to the content container). **Rules:** Resizing to fewer columns merges the removed columns' blocks into the last kept column — content is never deleted by a layout change; `stackOnMobile` (default true) drives the exported mobile media query. **Edge cases:** Resizing a 4-column row to 1 column concatenates all 4 columns' blocks into one, which can create a very long single column — accepted behavior, not flagged by Checks. **Error states:** None. **Dependencies:** F-001. **Acceptance criteria:**

```
Given a row with 3 columns each containing 1 block
When the layout is changed to 1 column
Then the resulting single column contains all 3 original blocks, in order, none lost
```

F-003 — Atomic content blocks

User story: As the editor, I want basic content primitives (heading, text, image, button, divider, menu, social, video, HTML), so that I can build any email layout from simple parts. **Trigger:** Drag or click-to-add from LeftPanel's Content tab. **Flow:** `addBlockSmart(type)` places the block near the current selection (or at the end of the last row, or creates a new row if the canvas is empty) → `createBlock(type)` applies type defaults. **UI behaviour:** Each type has its own RightPanel editor (`HeadingEditor`, `TextEditor` using TipTap rich text, `ImageEditor` with the library picker, `ButtonEditor`, `DividerEditor`, `MenuEditor`/`SocialEditor` repeaters, `VideoEditor`, `HtmlEditor` raw-HTML textarea). **Rules:** Image `alt` text is encouraged but not hard-blocked at the editor level (Checks flags a missing one as a warning, not an error, unless the field is required by a composite block — see F-004); all hrefs pass through `safeUrl()` sanitization at render time (only `https?:`, `mailto:`, `tel:`, `#`, `//`, `{{`, `*|`, or a `data:image/...;base64` pass; everything else is stripped to `''`). **Edge cases:** A Video block's thumbnail auto-derives for YouTube URLs; other platforms require a manually supplied thumbnail URL or the block renders with no preview image. **Error states:** None at the editor level; a malformed href is silently stripped by `safeUrl()` rather than erroring. **Dependencies:** F-016 (image picker), F-012 (Checks validates these fields). **Acceptance criteria:**

```
Given an Image block's href field contains "javascript:alert(1)"
When the email is rendered (canvas or export)
Then the href is stripped to an empty string — the script-URL never reaches the output
```

F-004 — Composite blocks

User story: As the editor, I want ready-made sections (header, hero, footer, article list) instead of hand-assembling them from atomic blocks every time. **Trigger:** Drag/click-to-add like any block; frequently arrive pre-populated via a preset (F-005). **Flow:** Each composite block builds a throwaway sub-block internally (e.g. a `menu` block for Header's nav) and calls the real atomic renderer on it — no duplicated HTML logic. **UI behaviour:** `HeaderEditor` offers logo-left/logo-center layouts and an "Extend background full width" switch (writes color onto the parent row); `HeroEditor` offers 5 layouts with card count following the layout (2 for `cards-2`, 3 for `cards-3`, auto-padded/trimmed on layout change); `FooterEditor` defaults address/legal/unsubscribe/social from the active brand with a per-email override and a "Use brand social" toggle; `ArticleListEditor` is a repeater (add/remove/move) capped at 10 items. **Rules:** A block can never paint edge-to-edge (confined to the content container) except via the Header/Footer "extend background" switch, which is last-writer-wins if two such blocks share a row; Hero's centered background image renders as `<td background>` with no VML, so Outlook on Windows shows the fallback color only; `alt` is a hard-enforced (red-asterisk, warned) field on composite-block images because inboxes block images by default. **Edge cases:** `showFooter` off (Email settings) drops Footer blocks from preview/export entirely while keeping them visible-but-dimmed on canvas for editing. **Error states:** None. **Dependencies:** F-003 (reused renderers), F-006 (brand pull-through for Header/Footer). **Acceptance criteria:**

```
Given a Hero block's layout is "cards-2" with 2 cards configured
When the layout is changed to "cards-3"
Then a 3rd card is added with default content, and the first 2 cards' content is preserved

Given Email settings → Show footer is off
When the email is exported
Then no Footer block's HTML appears in the output, while the canvas still shows it dimmed
```

F-005 — Starter presets

User story: As the editor, I want ready-made starting structures, so "New email" isn't a blank page. **Trigger:** TopBar "New" → NewEmailModal → Presets tab → pick a card. **Flow:** `preset.build(brand)` mints a fresh `Design` with new ids throughout → `applyEmailDefaults()` overlays org-wide defaults → `startNewEmail()` makes it the active document. **UI behaviour:** Each preset card shows a live thumbnail — the preset's real exported HTML rendered in a sandboxed, inert iframe at 600px, scaled down via CSS transform, themed with whichever brand is selected in the dialog header. **Rules:** Colors/fonts are brand tokens (re-theme on a later brand switch); the brand **name** is interpolated into prose once, at apply time, then becomes ordinary editable text; every preset ships placeholder art (`picsum.photos`) and placeholder/fictional copy **on purpose**, specifically so Checks' `IMAGE_PLACEHOLDER`/`LINK_PLACEHOLDER_DOMAIN` rules flag it until replaced. **Edge cases:** "Cold outreach" deliberately has zero images and no header/hero, reading as a personal email rather than a campaign. **Error states:** None — applying a preset cannot fail. **Dependencies:** F-004 (presets are built from composite + atomic blocks), F-006 (brand theming), F-012 (placeholder warnings). **Acceptance criteria:**

```
Given the Newsletter preset is applied twice in a row (as two separate new emails)
When their block ids are compared
Then every id is distinct between the two applications — neither aliases the preset's own
stored definition
```

F-006 — Brand kit & brand switcher

User story: As the editor, I want to theme any email instantly for a given brand, without touching its content. **Trigger:** Brands modal (create/edit/duplicate/archive); brand selector in TopBar or NewEmailModal. **Flow:** `upsertBrand()` saves kit fields → `setBrandId()` changes only `design.brandId` → `resolveDesign(design, brand, mode)` replaces every `brand.*` token with a literal value immediately before render/export. **UI behaviour:** BrandsModal: Identity, Logos (upload-preview pair + hosted-URL-export pair), Colors (9 named roles), Dark mode (derive/override), Fonts, Buttons, Footer, Social links, Voice (free-text guidance consumed only by F-018); a live kit preview panel. **Rules:** Switching brands never touches content or manual hex overrides (BR-02); exports always use the hosted `logo_url`, never the upload preview (BR-04); a `brandId` pointing at an archived/deleted brand silently falls back to the default brand (BR-03); resolution order for a blank field is active brand → seed brand → `''` ("inherit"). **Edge cases:** Archiving a brand (`is_active: false`) is a soft-delete — existing emails using it still resolve correctly; only active brands appear in pickers. **Error states:** "Give the brand a name first" if

saving with an empty name. **Dependencies:** F-007 (dark palette lives on the brand), F-005/F-018 (consume brand tokens/voice). **Acceptance criteria:**

```
Given an email has a manually-typed hex override on its heading color
When the active brand is switched
Then the heading color is unchanged, while every brand.*-token-sourced color re-themes
```

F-007 — Dark mode support

User story: As the editor, I want the export to look correct in dark-mode-aware inboxes, without maintaining two separate emails. **Trigger:** "Dark mode support" toggle (Email settings, default on); brand-level dark palette fields. **Flow:** `output.ts` resolves the design twice — once against the real brand, once against `darkBrand()` (dark-palette-swapped) — and diffs the two; a value that moved came from a token (gets a CSS class), a value that stayed was hand-typed (left untouched). **UI behaviour:** BrandsModal's Dark mode section: 8 fields, "Derive from light," "Reset to derived," a live sample, and a readable-contrast readout; TopBar/PreviewModal Light/Dark toggle (disabled when the email's dark mode is off). **Rules:** With the toggle off, export is byte-identical to a pre-dark-mode export (BR-22); a manual per-block/row dark override beats both the token-following rule and the derived palette; two hand-corrected role-mapping cases exist for ink-as-background and surface-as-foreground colors (BR-23). **Edge cases:** Gmail's own mail apps ignore all of this and apply their own automatic inversion (disclosed in the Preview UI, not something the app can override); a hero background image cannot be restyled for dark mode at all. **Error states:** None. **Dependencies:** F-006, F-012 (dark-specific contrast checks). **Acceptance criteria:**

```
Given Dark mode support is off
When the email is exported
Then the output contains no color-scheme meta tags, no dm- class names, and is byte-identical
to the same email with dark mode never having existed
```

F-008 — Merge tags

User story: As the editor, I want ESP-correct personalization placeholders, so the same design works regardless of which ESP sends it. **Trigger:** Insert-tag menu on the Text toolbar, Heading, Button label, Subject, Preheader. **Flow:** Content always stores canonical `{{tag}}`; `formatMergeTags()` rewrites the spelling only at export, per the chosen `mergeFormat`. **UI behaviour:** "Preview with sample data" swaps in sample values on canvas only (stored separately, never in the design); Email settings offers curly/Mailchimp/HubSpot/custom format choice with custom prefix/suffix fields. **Rules:** 6 built-ins (`first_name`, `last_name`, `company`, `email`, `unsubscribe_url`, `view_in_browser_url`) map to real ESP field syntax, not a naive transform; an unknown hand-typed tag falls back generically (uppercase for Mailchimp, `contact.<tag>` for HubSpot). **Edge cases:** Switching format never edits stored content, only the export. **Error states:** None. **Dependencies:** F-011 (export boundary). **Acceptance criteria:**

```
Given Email settings → Merge tag format is "Mailchimp"
When the email is exported
Then {{first_name}} in the design becomes *|FNAME|* in the output, and the stored design is
unchanged
```

F-009 — UTM link tagging

User story: As the editor, I want outbound links tagged for campaign tracking, applied consistently without manual editing of every link. **Trigger:** Email settings → UTM toggle; per-link "Don't add tracking" opt-out. **Flow:** `addUtm()` runs at export only, after brand resolution, while tags are still canonical (so "is this URL just a merge tag" stays one regex). **UI behaviour:** Email settings exposes source/medium/campaign/content/term fields (medium defaults to `email`; campaign defaults to a slug of the email title on first enable); the rich-text Link dialog has a "Don't add tracking" checkbox. **Rules:** Never overwrites an existing `utm_*` parameter; skips non-http(s) links, merge-tag-only URLs, and the unsubscribe/view-in-browser links (still counted in the "skipped" total); the opt-out attribute is stripped from the export regardless of whether UTM is globally on or off. **Edge cases:** A link containing both a merge tag and a real path is still evaluated for UTM eligibility correctly since UTM runs before tag reformatting. **Error states:** None. **Dependencies:** F-011. **Acceptance criteria:**

```
Given a link already has ?utm_source=newsletter on it
When UTM tagging runs at export
Then that parameter is left exactly as it is, not duplicated or overwritten
```

F-010 — Plain-text export

User story: As the editor, I want a text-only version alongside the HTML, for clients/recipients that need it. **Trigger:** Export dialog's "Plain text" tab. **Flow:** `toPlainText()` runs on the already-resolved design (post-UTM, post-merge-format). **UI behaviour:** Export dialog tabs (HTML | Plain text), each with its own Copy/Download. **Rules:** 72-character wrap; a URL longer than the wrap width is never split onto two lines; per-block-type rendering (headings on their own line, links as `text (url)`, buttons as `Label: url`, dividers as a dashed rule, Footer only if `showFooter` is on). **Edge cases:** An image with no alt text contributes nothing to the plain-text output (silently skipped, not an error). **Error states:** None. **Dependencies:** F-008, F-009 (shares the resolved pipeline). **Acceptance criteria:**

```
Given a Button block with label "Shop now" and href "https://example.com/shop"
When plain text is generated
Then the output line reads "Shop now: https://example.com/shop"
```

F-011 — HTML export & live preview

User story: As the editor, I want to produce the final HTML and see exactly what will ship before I export it. **Trigger:** TopBar "Export" / "Preview." **Flow:** `resolveForOutput(design, brand, mode)` →

`renderDesign()` → the string shown/downloaded; Preview renders the SAME string in a sandboxed iframe. **UI behaviour:** Export dialog shows size in KB, the UTM tagged/skipped count, and a one-line Checks summary with an "Open checks →" hand-off; Preview offers Desktop/Mobile and Light/Dark toggles. **Rules:** The canvas, Preview, and the final export are driven by the **same** renderer — "what you see is what is exported" is enforced by construction, not convention; Copy/Download are never disabled by outstanding Checks issues (BR-05). **Edge cases:** When the email contains unhosted AI drafts, Preview shows their real local bytes (something the export itself cannot do) and says so in a caption. **Error states:** "Copy failed — select the text and copy manually" if the clipboard API throws. **Dependencies:** F-006, F-007, F-008, F-009, F-012. **Acceptance criteria:**

```
Given an email with 1 outstanding Checks error
When the Export dialog is opened
Then Copy and Download remain clickable, and the dialog states "1 error" with a link to the
Checks drawer
```

F-012 — Checks panel (quality engine)

User story: As the editor, I want to know what an inbox will do with this email before I send it, without being blocked from exporting anyway. **Trigger:** Automatic (debounced 500ms on every design/brand change, and on a whole-design load); manual "Check images load." **Flow:** `checkDesign(resolvedDesign, exportedHtml, brand, opts)` returns `Issue[]`, grouped by severity. **UI behaviour:** TopBar badge (worst severity, or a green tick); drawer overlaying (not replacing) the RightPanel, with per-issue message/hint/"Go to block"/"Ignore"; an "Ignored (n)" section. **Rules:** Never blocks anything (BR-05); an image/draft is never called broken/missing until it has actually been checked (BR-07); issue ids are stable (`code#blockId#discriminator`) so "Ignore" persists meaningfully across unrelated edits and reloads. **Edge cases:** Text over a hero background image is reported `CONTRAST_UNKNOWN` rather than guessed at. **Error states:** None — the engine itself cannot fail the UI (a caller handing it an unresolved design is documented as a caller bug, not a runtime guard). **Dependencies:** Every content/export feature feeds this engine. **Acceptance criteria:**

```
Given an issue has been Ignored on a specific email
When the browser is reloaded and that email reopened
Then the issue stays hidden from the active list and appears under "Ignored"
```

F-013 — Saved blocks library

User story: As the editor, I want to save a block/row I built once and reuse it in any future email. **Trigger:** "Save as block" (RightPanel header on a selected block/row). **Flow:** `addSavedBlock(name, category, kind, payload)` freezes a JSON snapshot; `insertSavedBlock()` always clones with fresh ids before placing a copy on canvas. **UI behaviour:** LeftPanel's Saved tab: search, category grouping, per-item thumbnails rendered with the active brand, rename, delete, drag-or-click-to-insert; Export/Import JSON. **Rules:** A `data:` image embedded in the payload triggers a save-time warning (won't survive a real inbox); import skips exact-id duplicates silently and asks before overwriting a name clash. **Edge cases:** Deleting a library entry never affects emails that already used it (payload was copied at insert

time). **Error states:** "Could not read that file" / "That file is not a saved-blocks JSON" on a malformed import. **Dependencies:** F-001, F-023 (export/import mechanics). **Acceptance criteria:**

```
Given a saved block containing brand.primary as a color token
When it is inserted into an email themed with a different brand than the one it was saved
under
Then the block renders in the new brand's primary color, not the original brand's
```

F-014 — Auto-save

User story: As the editor, I never want to lose work by closing a tab. **Trigger:** Any design edit (2s debounce); tab close (`beforeunload` flush). **Flow:** First edit creates a `SavedTemplate` record (`autoSaved: true`); subsequent edits update it. **UI behaviour:** TopBar status: "Saving…" → "Saved just now" → "Saved 5 min ago" (30s tick) → "Not saved: storage full" on failure. **Rules:** A whole-design **load** (open/restore/undo) never marks the email dirty — only a real edit does (BR-11); on quota pressure, the 10 globally-oldest version snapshots are pruned and the write retried before reporting failure (BR-13). **Edge cases:** Picking a preset or starting blank never auto-saves over the email just left (`startNewEmail` leaves `currentEmailId: null` until the next edit). **Error states:** "Not saved: storage full" is never silent. **Dependencies:** F-015 (shares the storage quota-recovery path). **Acceptance criteria:**

```
Given a brand-new, never-saved email
When 2 seconds pass after the very first edit
Then a SavedTemplate record exists, grouped under "Drafts (auto-saved)" until explicitly
renamed
```

F-015 — Version history

User story: As the editor, I want to go back to an earlier version of this email without losing my current state. **Trigger:** Automatic (≤1/5min of active editing); manual "Save version"; pre-restore/pre-replace. **Flow:** `shouldSnapshot()` decides whether to snapshot (never if identical to the latest); `pruneToCap()` keeps the newest 20 per email. **UI behaviour:** HistoryModal: optional note field, "Save version," a list with relative time + row/block counts, Preview (read-only iframe), Restore. **Rules:** Restoring snapshots the present design first, so a restore is itself always reversible (BR-12); versions store the design only — never brand definitions. **Edge cases:** "Save version" right after an auto-save can be a no-op ("No changes since the last version"). **Error states:** "Version not saved: storage full." **Dependencies:** F-014. **Acceptance criteria:**

```
Given a version is restored
When the restore completes
Then the design present immediately before the restore now exists as its own version,
restorable forward again
```

F-016 — Image library (URL-based)

User story: As the editor, I want a vetted, reusable library of image URLs instead of re-typing/re-finding the same image every time. **Trigger:** LeftPanel Images tab "Add"; any image field's picker button. **Flow:** `validateImageUrl()` (https-only) → `probeImage()` (does it actually load) → `makeLibraryImage()` stores name/alt/tags/brand scope/measured size. **UI behaviour:** Grid of thumbnails with a "Could not load" badge when broken; search; This brand/Shared/All filter; Export/Import JSON. **Rules:** File "Upload from device" is **categorically refused** with an explicit message (BR-17) — it never falls back to an embedded `data:` URI; resolution notes ("Low resolution"/"Very large image") are advisory only and require the image to already be in the library (where its measured size lives). **Edge cases:** The library re-probes every item on open (fine for tens of images; would need batching at hundreds — a stated known gap). **Error states:** A pasted URL that fails validation shows the specific rejection reason inline (not https / malformed / empty). **Dependencies:** F-012 (shares the audit/classification logic). **Acceptance criteria:**

```
Given a user pastes an http:// (not https) image URL
When they attempt to add it to the library
Then it is rejected with a reason, and nothing is added
```

F-017 — AI image generation

User story: As the editor, I want to generate an on-brand image from a text prompt, without leaving the editor or using a separate tool. **Trigger:** Any image field's "Generate with AI" entry point. **Flow:** One request to OpenAI's image endpoint (`openai.ts`) → magic-byte response validation (`validate.ts`) → bytes stored in IndexedDB as a `DraftImage`; the design field stores only an `aiDraftId`. **UI behaviour:** AiImagePanel: prompt composer, aspect/style/quality selectors, a "Brand colours" toggle, a chat-like turn history, Cancel/Regenerate/Edit prompt/Download PNG per result; alt text required before "Add to email." **Rules:** Exactly one request per generation, never auto-retried (BR-06); the API key defaults to in-memory-only and is never serialized into any exportable state (BR-08); because no image hosting exists, every export of a generated image ships a visible placeholder URL until manually rehosted. **Edge cases:** `AI_IMAGE_MISSING_DRAFT` (bytes not in this browser — e.g. an imported design) vs. `AI_IMAGE_NOT_HOSTED` (bytes present locally but nowhere public) are distinct, mutually exclusive Checks rules, both reported only after the app has actually looked. **Error states:** Classified, redacted error cards (invalid key, forbidden, no credit, rate-limited, content-policy refusal, network/gateway timeout with a "may have already been billed" warning, or a generic bad response) — the raw provider error text is never shown un-redacted. **Dependencies:** F-019/F-020/F-021 (routing, fallback, key management), F-012 (drafts-specific checks). **Acceptance criteria:**

```
Given a generation request fails with a 503 status
When the error is shown
Then the message warns the image may already have been produced and billed, and no automatic
retry occurs
```

F-018 — Generate email from a brief (AI)

User story: As the editor, I want to turn a short content brief into a first-draft email, reviewed by me before anything is finalized. **Trigger:** NewEmailModal → "Generate from brief" tab. **Flow:** `buildSlotSchema()` walks the resolved preset's real block tree and classifies each block's fields by type (the "slot-derivation pattern") → one strict-JSON-schema request → `brief-validate.ts` binary accept/reject → `BriefReview` (no design mutation yet) → "Create email" writes onto a fresh design copy. **UI behaviour:** BriefForm: topic*, audience, goal, up to 8 key points, tone, length, CTA label/URL, article count, language, preset (or Auto), optional source text (≤3000 chars), a one-time data-use consent notice per provider; BriefReview: subject-option buttons + editable subject, read-only preheader, copyable image-prompt suggestions, leftover-`[bracket]`-placeholder warnings, Create/Regenerate/Edit-brief. **Rules:** A field is AI-fillable exactly when the preset's own placeholder copy for it is non-empty; AI is categorically excluded from footer/legal copy, structural blocks, logos, and the hero background image; a slot is exposed only if the preset's actual **rendering** of that field would be true (hero-card slots only on `cards-2`/`cards-3` layouts) — this is the fix for a real bug found during development (BR-26); a user-supplied CTA removes the corresponding AI slot(s) entirely (BR-27); validation is binary — the whole response is accepted or rejected, never partially repaired (BR-28). **Edge cases:** "5B"/"5C" and a literal "4A" feature referenced in early planning notes do not exist anywhere in this codebase — any requirement citing them should be treated as referring to 5A's own amendments (Not Found in Codebase as standalone features). **Error states:** A rejected AI response surfaces its reason with a "Regenerate" affordance; nothing is written to any design until "Create email." **Dependencies:** F-005 (the structural target), F-006 (brand voice input), F-012 (`AI_DRAFT_UNREVIEWED`, `BRIEF_PLACEHOLDER_LEFT`), F-019/F-020/F-021. **Acceptance criteria:**

```
Given a brief with a non-empty CTA label
When the AI schema is built
Then every cta_label-kind slot is absent from it, and the user's own label/URL are written
directly after Create

Given an email was created from a brief
When it is opened
Then the Checks drawer shows AI_DRAFT_UNREVIEWED until "Mark as reviewed" is clicked
```

F-019 — Per-task AI model routing

User story: As the editor, I want text generation and image generation to use independently chosen models. **Trigger:** Settings → AI → "Models per task." **Flow:** `runTask(taskId, input)` reads the task's configured route (provider, model, options) rather than any global "active provider." **UI behaviour:** One row per task (`text.brief`, `image.generate`): Provider select, Model select or a free-text "Other model id," the task's own quality/temperature option, a status chip, "Test connection," its own fallback-chain editor. **Rules:** A per-generation Quality selector in the AI image composer always wins over the task's configured default quality — routing options are defaults, not a replacement for a choice the UI already exposes per generation; "Reset to defaults" restores a deliberately-chosen model (`gpt-4.1-mini` / `gpt-image-2`), not whichever is newest in the list. **Edge cases:** On first-ever use, `image.generate`'s route is seeded from whatever the pre-6B single "active provider" held (one-time migration, then frozen — not a live mirror thereafter). **Error states:** "No access to `<model>`" status chip with a hint to choose another

model or adjust OpenAI project settings. **Dependencies:** F-017, F-018, F-020, F-021. **Acceptance criteria:**

```
Given a task's configured default quality is "High"
When the user picks "Draft" for one specific generation
Then that one generation uses "Draft" — the task's stored default is not changed
```

F-020 — Provider/model fallback chains

User story: As the editor, I want a configured backup model so one provider hiccup doesn't block my work. **Trigger:** "Add fallback" in a task's routing row. **Flow:** On a failed primary call, `run-task.ts` tries each fallback entry in order, skipping any with no configured/connected key. **UI behaviour:** An ordered list of Provider+Model pickers per task, each removable. **Rules:** A fallback is tried **only** for `network`/`timeout`/`rate-limit`-classified failures; any other failure (bad key, no credit, content-policy, malformed response) stops immediately — "a different model on the same account wouldn't fix any of those" (BR-29). **Edge cases:** Today there is exactly one registered AI provider, so the fallback chain is "same provider, different model" in practice; the data model supports multiple providers for when a second exists. **Error states:** If every candidate (primary + all fallbacks) fails or is skipped, the original failure is surfaced. **Dependencies:** F-017, F-018, F-019. **Acceptance criteria:**

```
Given a task has one fallback entry with a connected key
When the primary call fails with a network timeout
Then the fallback entry is tried automatically, with no user action required
```

F-021 — AI provider & API key management

User story: As the editor, I want to connect my own OpenAI key safely, and know whether it's actually working. **Trigger:** AiImagePanel gear icon, or Settings → AI (same underlying component). **Flow:** "Test and save" only persists a key after a passing connection test; a failed attempt leaves the previously working key untouched (BR-30). **UI behaviour:** Masked key display; show/hide toggle; "Remember on this device" checkbox (explicitly labeled "Stored unencrypted in this browser, for this provider only"); status badge (Not configured / Configured / Connected). **Rules:** The key defaults to in-memory-only (dies with the tab); it is deliberately excluded from the app's state store and from backups, both structurally (the relevant files never import the key-store module) and by dedicated tests sweeping every export/backup/saved-block surface (BR-08); provider error text is redacted (`redactKeys()`) before ever reaching the UI. **Edge cases:** Removing the active provider's key falls back to the first other Connected provider, else no provider is active and dependent composers show a setup prompt instead of being usable. **Error states:** Connection-test failures surface the provider's own (redacted) message plus a compact diagnostic (status/content-type/timing), never the key itself. **Dependencies:** F-017, F-018, F-019, F-020. **Acceptance criteria:**

```
Given a stored, working key
When a replacement key is entered and fails Test connection
Then the original key remains the active, stored one — nothing is overwritten
```

F-022 — Settings hub & profile

User story: As the editor, I want one place to manage my profile, AI setup, brand defaults, and local data, instead of hunting across the app. **Trigger:** LeftPanel's Settings gear or Profile row. **Flow:** A two-level navigation (section list → section detail); Escape backs up one level at a time. **UI behaviour:** 7 sections (Profile, AI, Brands, Email defaults, Checks, Data & storage, About); on narrow viewports the hub becomes a full-screen sheet (the only responsive behavior in the app's own chrome). **Rules:** "Email defaults" apply only to a brand-new email at the moment of creation, never to an already-open/saved email (BR-14); the Profile display name is a local, unauthenticated label used only for `createdBy`/`updatedBy` attribution (BR-10) — not a login. **Edge cases:** Opening the hub snapshots the previously active left-panel tab and scroll position, restoring both on close. **Error states:** "A display name is required" if the Profile field is emptied and saved. **Dependencies:** Surfaces F-006, F-008, F-009, F-007, F-012, F-019–F-021, F-023 — it is a shell over those, not independent logic. **Acceptance criteria:**

```
Given the left panel was on the "Saved" tab, scrolled partway down
When Settings is opened and then closed without changing tabs
Then the left panel returns to "Saved," at the same scroll position
```

F-023 — Import / export & backup

User story: As the editor, I want to move my brands/templates/blocks to another browser, and recover or wipe my local data deliberately. **Trigger:** TopBar Import/Download JSON (per-email design); BrandsModal Export/Import; Saved-blocks/Image-library Export/Import; Settings → Data & storage (full backup/restore, clear-all). **Flow:** `normalizeDesign()` is the single gate for loading any JSON (missing fields filled from current defaults; unknown block types dropped; never crashes on malformed input); `exportBackup()`/`importBackup()` cover brands+templates+saved blocks+images+versions as one JSON file. **UI behaviour:** Backup import shows a record-count summary before anything is overwritten; "Clear all local data" requires literally typing `DELETE` before the button enables. **Rules:** Backup structurally excludes the AI key store and IndexedDB drafts (the file never imports either module) (BR-15); a malformed individual record within an import is dropped, not a reason to fail the whole import; "Clear all local data" is the one action allowed to touch both the AI key store and drafts, specifically to delete them (BR-16). **Edge cases:** A design imported from another browser that references an AI draft will show `AI_IMAGE_MISSING_DRAFT` — a normal state, not an error, since drafts never travel with a design. **Error states:** "That file is not a valid email design JSON" / "...brand kit JSON" / "...saved-blocks JSON" / "...image library JSON" on malformed input; "Could not read that backup: `<reason>`" on a malformed backup file. **Dependencies:** F-006, F-013, F-016, F-017 (interaction with drafts on import), F-021 (explicitly excluded from this feature's scope). **Acceptance criteria:**

```
Given a backup JSON file with a corrupted single template record but valid brands/images
When it is imported
Then the corrupted template is dropped and everything else imports successfully — the whole
import is not rejected

Given the "Clear all local data" field does not contain exactly the word DELETE
When the user attempts to click "Clear all local data"
Then the action is disabled
```

Part 3 of 4 — Sections 6–10: User Flows, Screens, Functional Requirements, NFRs, Data

> Continues from `PRD-02-feature-specs.md`. `PR-xxx` ids introduced in §8 are used again, unchanged, in Part 4 §15 (traceability matrix).

---

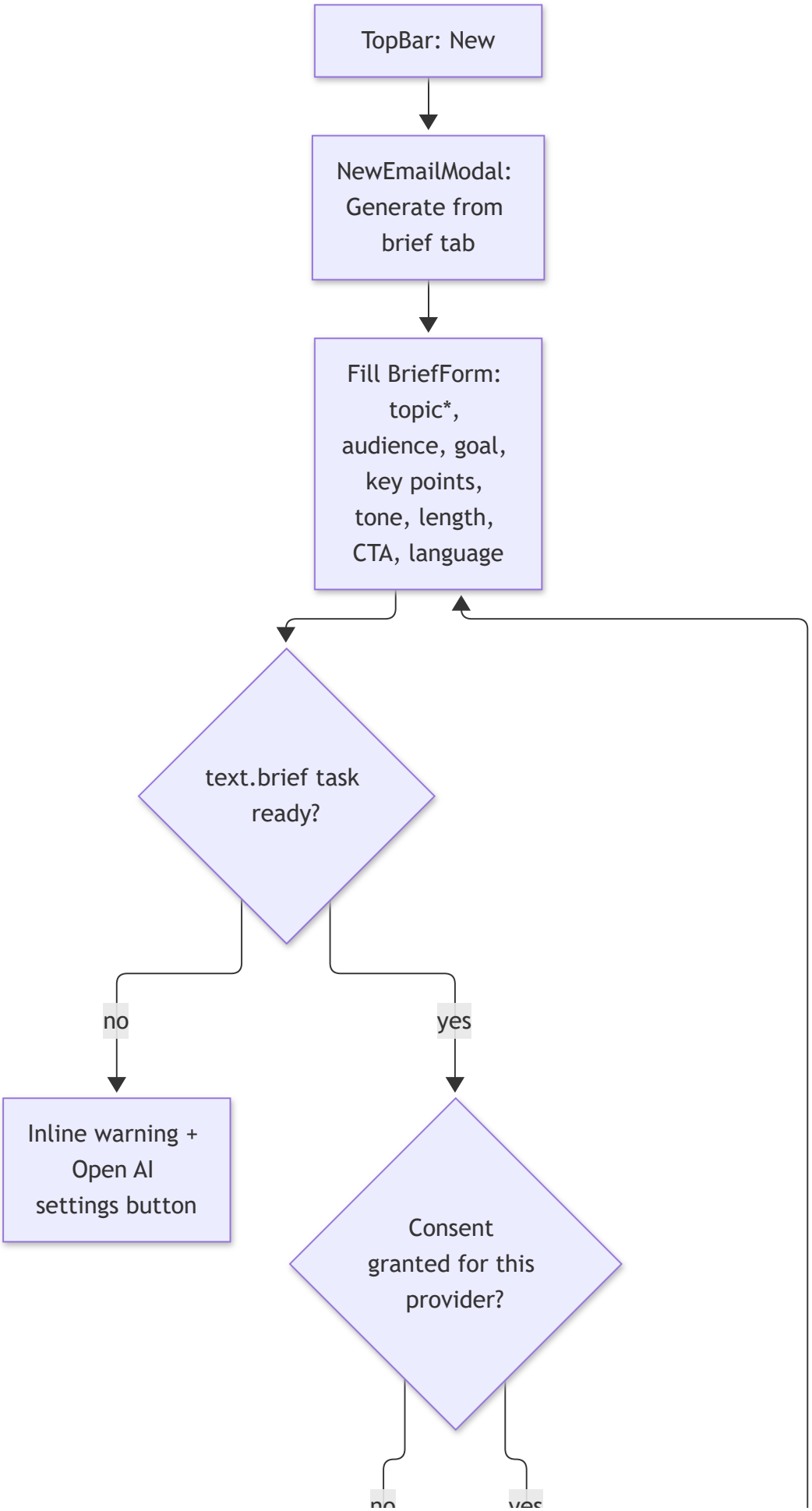
6. User Flows

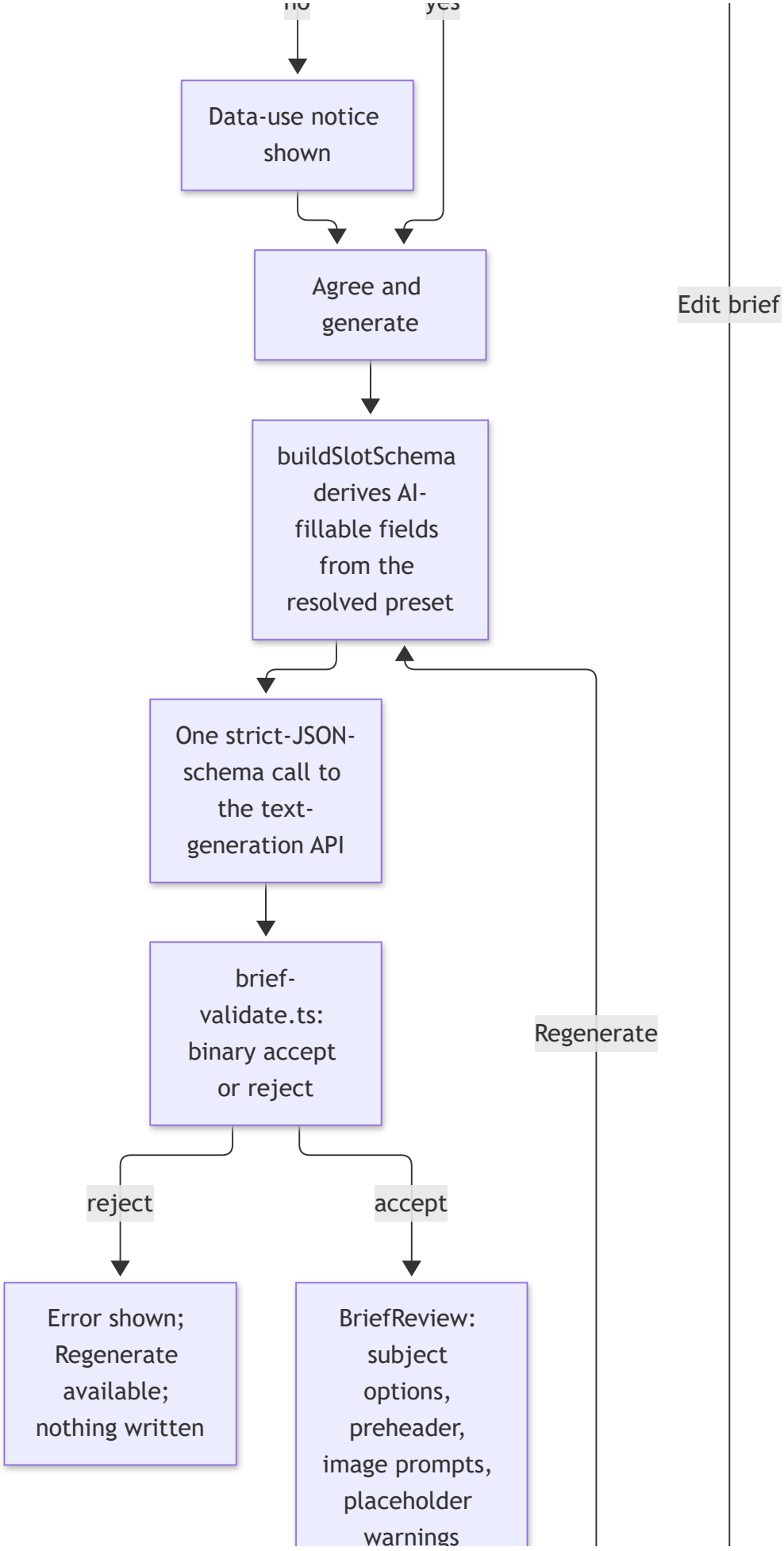
6.1 New email from a preset

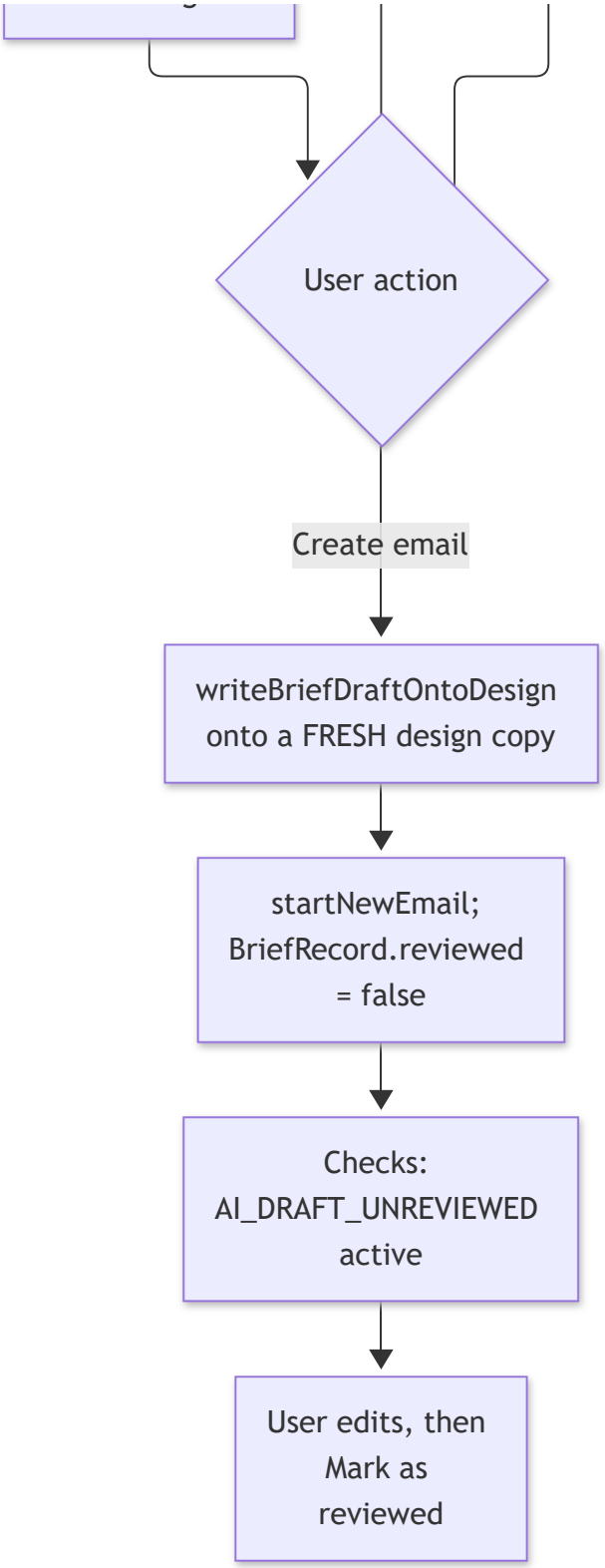
Diagram could not be rendered (Parse error on line 6:).

```
flowchart TD
    A[TopBar: New] --> B[NewEmailModal opens, Presets tab]
    B --> C[Pick a brand in the dialog header]
    C --> D[Thumbnails re-render themed with that brand]
    D --> E{Pick Blank or a preset card}
    E -->|dirty design exists| F[Confirm: "A change from the last couple of seconds may not
be saved yet"]
    F -->|confirmed| G
    E -->|not dirty| G[snapshot pre-replace; preset.build(brand) or emptyDesign()]
    G --> H[applyEmailDefaults overlays org-wide defaults]
    H --> I[startNewEmail - currentEmailId = null]
    I --> J[Canvas shows the new design; modal closes]
    J --> K[First edit triggers auto-save, minting a new SavedTemplate]
```

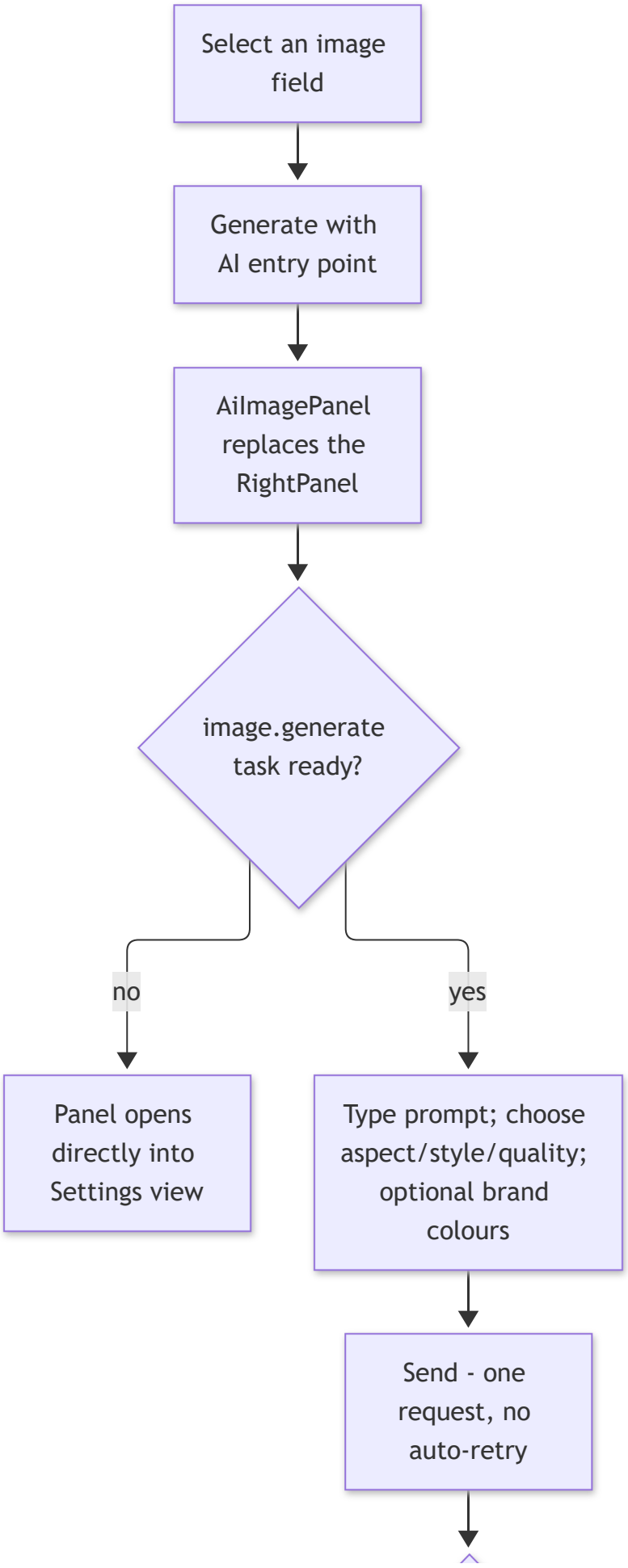
6.2 New email from a brief (AI)

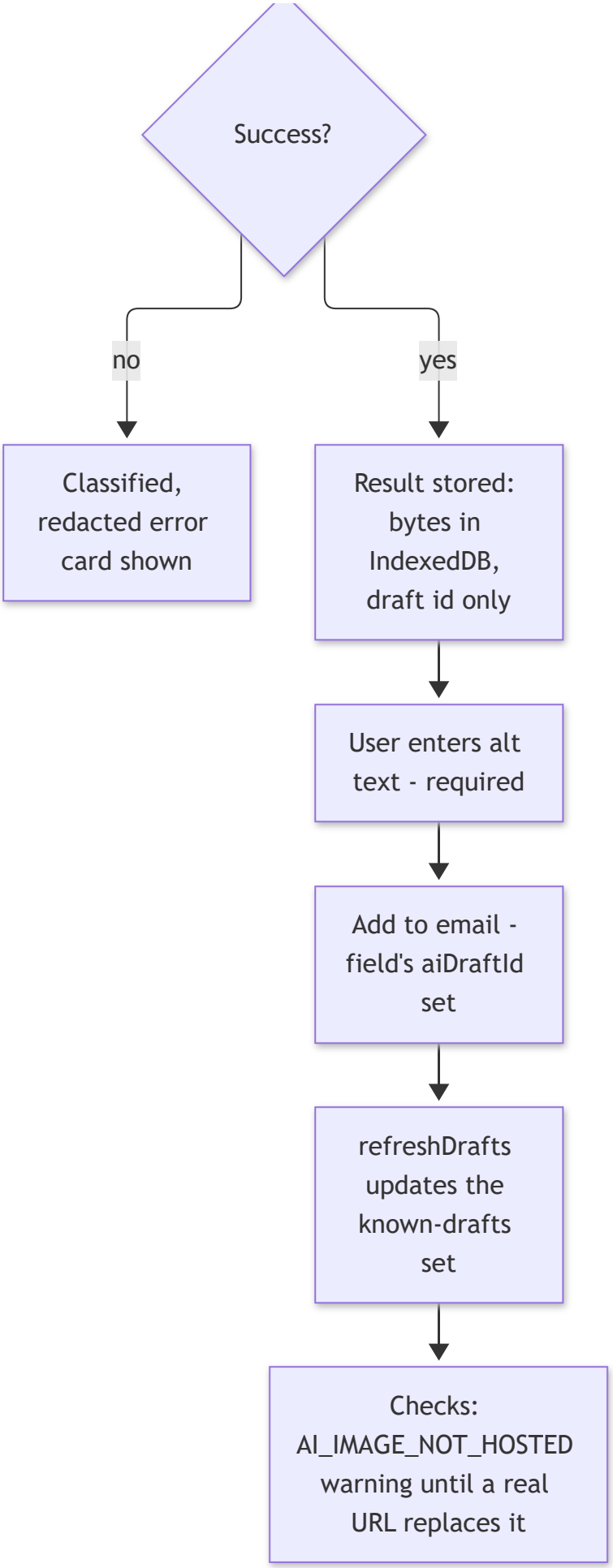

TopBar: New
NewEmailModal: Generate from brief tab
Fill BriefForm: topic*, audience, goal, key points, tone, length, CTA, language
text.brief task ready?
no
yes
Inline warning + Open AI settings button
Consent granted for this provider?


no
yes
Data-use notice shown
Agree and generate
Edit brief
buildSlotSchema derives AI-fillable fields from the resolved preset
One strict-JSON-schema call to the text-generation API
brief-validate.ts: binary accept or reject
Regenerate
reject
accept
Error shown; Regenerate available; nothing written
BriefReview: subject options, preheader, image prompts, placeholder warnings

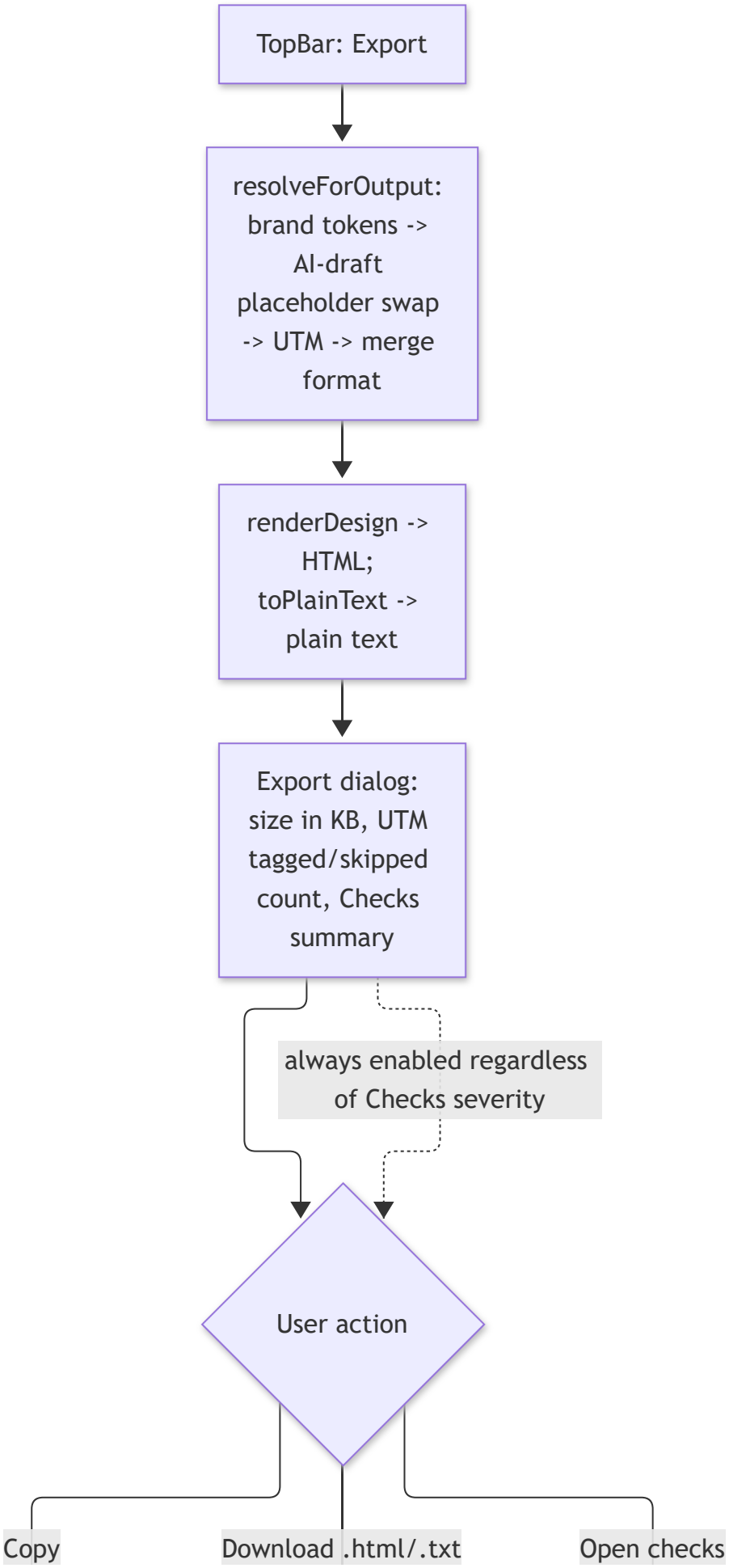


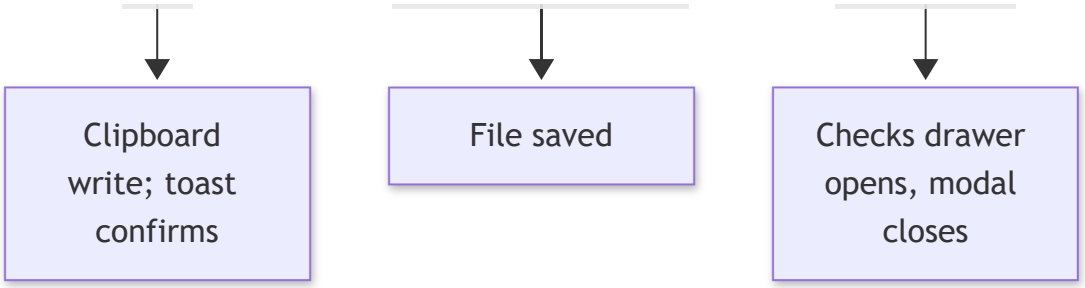
6.3 AI image generation and use


Select an image field
Generate with AI entry point
AiImagePanel replaces the RightPanel
image.generate task ready?
no
yes
Panel opens directly into Settings view
Type prompt; choose aspect/style/quality; optional brand colours
Send - one request, no auto-retry

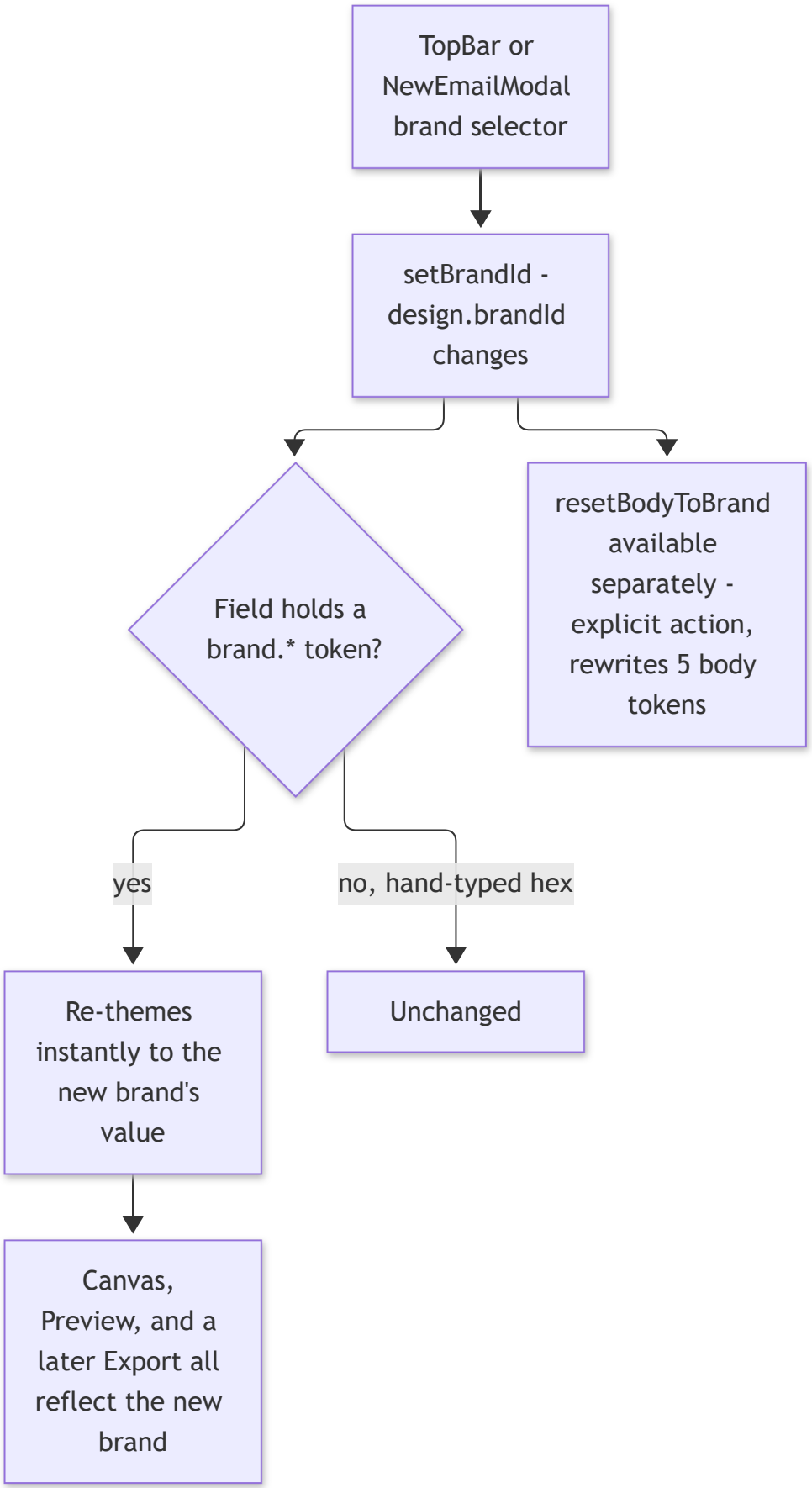

Success?
no
yes
Classified, redacted error card shown
Result stored: bytes in IndexedDB, draft id only
User enters alt text - required
Add to email - field's aiDraftId set
refreshDrafts updates the known-drafts set
Checks: AI_IMAGE_NOT_HOSTED warning until a real URL replaces it

6.4 Export

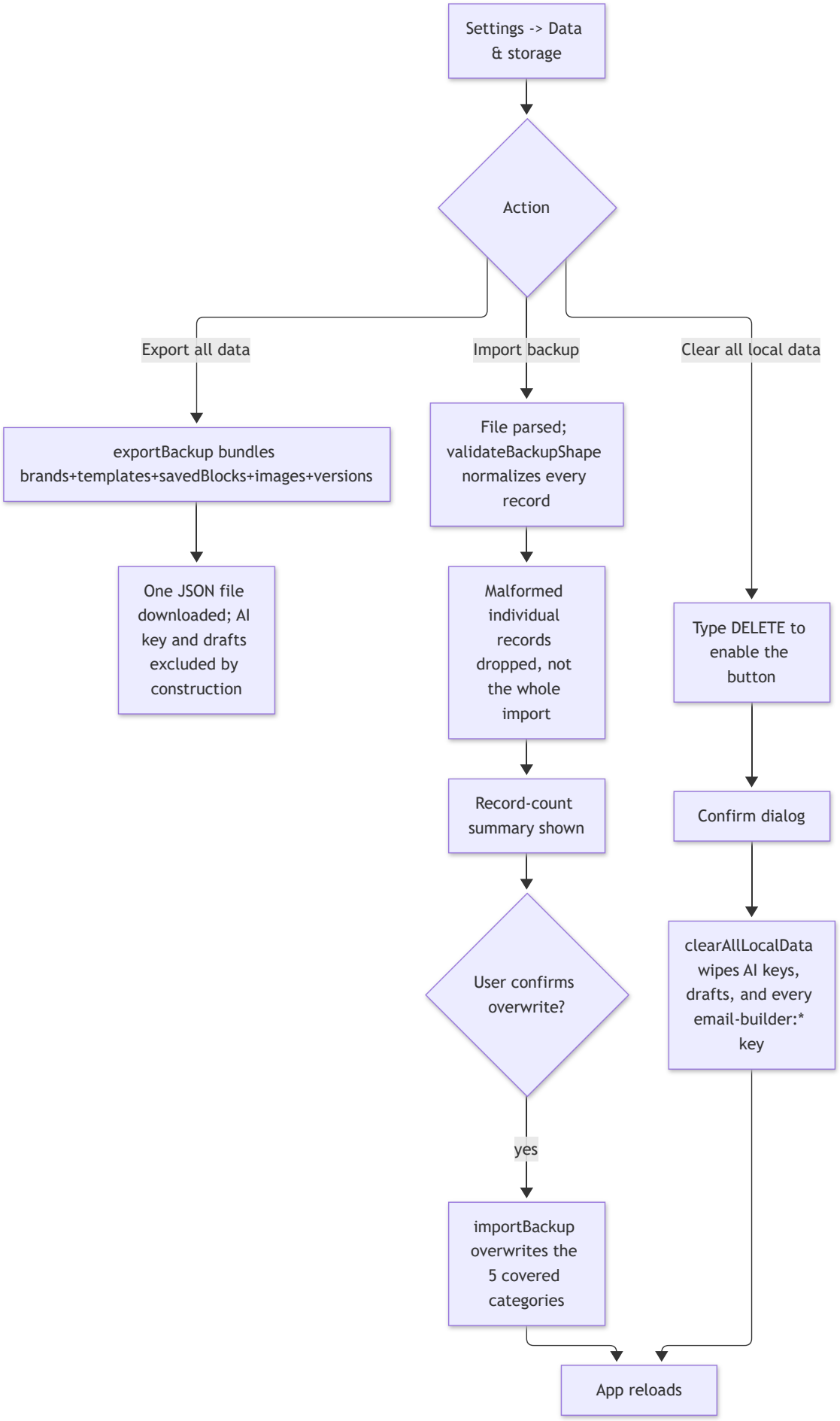

TopBar: Export
resolveForOutput: brand tokens -> AI-draft placeholder swap -> UTM -> merge format
renderDesign -> HTML; toPlainText -> plain text
Export dialog: size in KB, UTM tagged/skipped count, Checks summary
always enabled regardless of Checks severity
User action
Copy
Download .html/.txt
Open checks



6.5 Brand switch


TopBar or NewEmailModal brand selector
setBrandId - design.brandId changes
Field holds a brand.* token?
resetBodyToBrand available separately - explicit action, rewrites 5 body tokens
yes
no, hand-typed hex
Re-themes instantly to the new brand's value
Unchanged
Canvas, Preview, and a later Export all reflect the new brand

6.6 Import / backup


Settings -> Data & storage
Action
Export all data
Import backup
Clear all local data
exportBackup bundles brands+templates+savedBlocks+images+versions
One JSON file downloaded; AI key and drafts excluded by construction
File parsed; validateBackupShape normalizes every record
Malformed individual records dropped, not the whole import
Record-count summary shown
User confirms overwrite?
yes
importBackup overwrites the 5 covered categories
Type DELETE to enable the button
Confirm dialog
clearAllLocalData wipes AI keys, drafts, and every email-builder:* key
App reloads

7. Screen and Panel Inventory

No URL router exists anywhere in the app (confirmed: `App.tsx` uses a `ModalName` union and component state, no `react-router`/history API). Every "screen" below is a modal, an overlay, or a panel swap.

Screen/Panel	Purpose	Entry point(s)	Key controls	States
TopBar	Title, brand switch, undo/redo, device/scheme toggle, save status, modal launchers	Always visible	New, Templates, History, Import/Download JSON, Export, Checks badge, Preview	Save status (idle/saving/saved/error); undo/redo enabled/disabled
LeftPanel	Add content; manage saved blocks/images; Settings/Profile entry	Always visible	4 tabs (Content/Rows/Saved/Images); Settings gear; Profile row	Tab selection; Settings-hub-open (replaces tab content); narrow-viewport full-screen sheet
Canvas	WYSIWYG editing surface = the real export renderer	Always visible	Click/drag select, drag-reorder, per-block duplicate/delete	Selected block/row highlighted; hidden Footer dimmed with "Hidden on send" badge
RightPanel	Property editor for the current selection, or email-wide settings	Always visible (replaced by AiImagePanel during image generation)	Per-type editors; Save as block/Duplicate/Delete; Reset to brand	Empty-selection (body settings) vs. block vs. row
NewEmailModal	Start a new email	TopBar "New"	Brand selector; Presets/Brief tabs	Presets grid vs. Brief-form vs. Brief-review
BriefForm	Collect AI brief inputs	Inside NewEmailModal	Topic*, audience, goal, key points, tone, length, CTA, language, preset, source text	Not-ready (no key) / consent-pending / ready-to-generate / busy
BriefReview	Human review of the AI draft	After a successful generation	Subject picks, editable subject, Create/Regenerate/Edit-brief	Busy (regenerating) vs. idle
BrandsModal	Brand-kit CRUD	TopBar palette icon; Settings → Brands	New/Import; per-brand Export/Duplicate/Edit/Archive/Restore	List view vs. edit-form view
TemplatesModal	Browse/open/rename/delete saved emails	TopBar "Templates"	Rename this email; Save as a copy; Open; Delete	Named templates vs. "Drafts (auto-saved)" grouping

Screen/Panel	Purpose	Entry point(s)	Key controls	States
HistoryModal	Version browse/restore	TopBar "History"	Save version; Preview; Restore	List view vs. read-only version-preview view
ExportHtmlModal	Produce the deliverable	TopBar "Export"	Copy, Download, Open-checks	HTML tab vs. Plain-text tab
PreviewModal	See the real exported HTML	TopBar "Preview"	Desktop/Mobile; Light/Dark	Dark toggle disabled when the email's dark mode is off
ChecksPanel (drawer)	Review/act on quality issues	Checks badge; "Open checks"	Go to block; Ignore/Un-ignore; Check images load	Empty (all clear) vs. grouped-by-severity list; "Ignored (n)" collapsed/expanded
AiImagePanel	Generate/use an AI image	Any image field's AI entry point	Prompt, aspect, style, quality, brand-colours; Send/Cancel/Regenerate/Edit-prompt/Download	Chat view vs. Settings view; composer enabled/disabled by readiness
AI settings view (shared)	Manage provider key(s)	AiImagePanel gear; Settings → AI	Add/Replace key, Test and save, Test connection, Remove key	Per-provider: Not configured / Configured / Connected
ImageLibrary / ImagePickerModal	Manage/choose vetted image URLs	LeftPanel "Images" tab; any image field's picker	Add (URL paste + probe), Export, Import, Copy URL, Delete	Browse mode vs. pick mode; per-image ok/broken/unknown
SaveBlockDialog	Save a block/row to the library	"Save as block"	Name*, Category	—
SettingsHub	Single configuration home	LeftPanel gear/Profile	7 section rows	Section list vs. section detail
Settings → Profile	Local display name	SettingsHub	Display name, Save	Valid/invalid (empty)
Settings → AI	Keys + per-task routing	SettingsHub	Per-task provider/model/option/fallback pickers, Test connection, Reset to defaults	Status chip per task; denial chip
Settings → Brands	Default brand + management entry	SettingsHub	Default-brand dropdown, "Manage brands"	—

Screen/Panel	Purpose	Entry point(s)	Key controls	States
Settings → Email defaults	Org-wide new-email defaults	SettingsHub	Merge format(+custom), UTM source/medium, Dark-mode toggle	—
Settings → Checks	Bulk-reset dismissed warnings	SettingsHub	Reset for this email (n); Reset for every email	—
Settings → Data & storage	Backup/restore/clear	SettingsHub	Export all data; Import backup; typed-`DELETE` danger zone	Pending-import summary vs. idle; danger zone collapsed/expanded
Settings → About	Version + shortcuts reference	SettingsHub	Link to README	—
ConfirmDialog	Reusable destructive-action confirm	Any destructive action	Confirm/Cancel	Danger-styled vs. neutral

8. Functional Requirements (PR ids)

Priority is **Inference** (MoSCoW) — no prioritization document exists; a requirement is Must if removing it would break a BR-cited guarantee or a core authoring workflow, Should if it is a documented, deliberately-engineered behavior that is not load-bearing for correctness, Could if it is a UX nicety, Won't if it is explicitly out of scope today (listed for completeness, not as a demand).

PR id	Requirement	Feature	Priority	BR link
PR-001	The system shall render every block through one shared function used identically by the canvas and the export	F-001	Must	no BRD link
PR-002	The system shall provide undo/redo with at least 100 steps and coalesce rapid same-field edits into one step	F-001	Should	no BRD link
PR-003	The system shall support rows with 1–4 columns and arbitrary ratio layouts	F-002	Must	no BRD link
PR-004	A row layout change shall never delete content; overflow blocks merge into the last kept column	F-002	Must	no BRD link
PR-005	The system shall provide 9 atomic block types: heading, text, image, button, divider, menu, social, video, HTML	F-003	Must	no BRD link
PR-006	Every href/src rendered to HTML shall pass through scheme sanitization (`safeUrl`)	F-003	Must	no BRD link

PR id	Requirement	Feature	Priority	BR link
PR-007	The system shall provide 4 composite block types (Header, Hero, Footer, Article list) built from the atomic renderers, not duplicated HTML	F-004	Must	no BRD link
PR-008	Composite-block images shall require alt text in the editor	F-004	Should	no BRD link
PR-009	Hiding the Footer (Show footer off) shall remove it from preview/export while keeping it visible-but-dimmed on canvas	F-004	Should	no BRD link
PR-010	The system shall offer 8 preset structures plus Blank, each re-themeable per brand	F-005	Must	BR-24
PR-011	Applying a preset shall mint fresh ids so the applied copy can never alias the stored preset	F-005	Must	BR-25
PR-012	Preset art/copy shall ship as placeholder on purpose, flaggable by Checks until replaced	F-005	Should	no BRD link
PR-013	The system shall support named, CRUD-able brand kits (colors, fonts, logos, footer, social, voice)	F-006	Must	no BRD link
PR-014	Switching the active brand shall change only `design.brandId`; content and manual overrides shall be untouched	F-006	Must	BR-02

PR id	Requirement	Feature	Priority	BR link
PR-015	A design referencing an archived/deleted brand shall silently resolve to the default brand	F-006	Must	BR-03
PR-016	Export shall use only the hosted logo URL, never an uploaded preview logo	F-006	Must	BR-04
PR-017	The system shall support a per-brand dark palette, derivable automatically when unset	F-007	Should	no BRD link
PR-018	With dark mode off, export shall be byte-identical to a build with no dark-mode feature at all	F-007	Must	BR-22
PR-019	A color sourced from a brand token shall move between light/dark passes; a hand-typed hex shall not	F-007	Must	BR-23
PR-020	The system shall store merge tags canonically as `{{tag}}` and reformat only at export	F-008	Must	BR-21
PR-021	The system shall support at least 4 merge-tag output formats (curly, Mailchimp, HubSpot, custom)	F-008	Should	no BRD link
PR-022	UTM tagging shall apply only at export, never editing stored content	F-009	Must	no BRD link
PR-023	UTM tagging shall never overwrite an existing `utm_*` parameter	F-009	Must	BR-19

PR id	Requirement	Feature	Priority	BR link
PR-024	The unsubscribe/view-in-browser links shall never be UTM-tagged, but shall be counted in the skipped total	F-009	Should	BR-20
PR-025	The system shall generate a plain-text alternative from the same resolved design as the HTML	F-010	Must	no BRD link
PR-026	Plain-text output shall never split a URL across the wrap width	F-010	Should	no BRD link
PR-027	The Export dialog shall report export size and UTM tagged/skipped counts	F-011	Should	no BRD link
PR-028	Export/Copy/Download shall remain enabled regardless of outstanding Checks severity	F-011	Must	BR-05
PR-029	Preview shall render the literal string that export would produce, with no second renderer	F-011	Must	no BRD link
PR-030	The system shall evaluate size, link, subject/preheader, contrast, readability, content, and image rules on every design/brand change	F-012	Must	no BRD link
PR-031	An image or AI draft shall never be reported broken/missing until it has actually been checked	F-012	Must	BR-07

PR id	Requirement	Feature	Priority	BR link
PR-032	A dismissed ("Ignored") issue shall persist per email and survive reload until explicitly un-ignored	F-012	Should	no BRD link
PR-033	The system shall support saving any block or row to a named, categorized, reusable library	F-013	Must	no BRD link
PR-034	Inserting a saved block shall always mint fresh ids, never aliasing the library original	F-013	Must	BR-25
PR-035	The system shall auto-save every design change within a few seconds, creating a record on first edit	F-014	Must	BR-11
PR-036	A save failure shall always be surfaced to the user, never silent	F-014	Must	no BRD link
PR-037	On storage-quota pressure, the system shall prune old version history automatically before reporting failure	F-014	Should	BR-13
PR-038	The system shall keep up to 20 version snapshots per email, newest-prioritized	F-015	Must	BR-13
PR-039	Restoring a version shall snapshot the current state first, making the restore itself reversible	F-015	Must	BR-12

PR id	Requirement	Feature	Priority	BR link
PR-040	The system shall never snapshot a design identical to the latest stored version	F-015	Should	BR-12
PR-041	The system shall maintain a reusable library of vetted https image URLs with name/alt/tags/brand scope	F-016	Must	no BRD link
PR-042	File upload for content images shall be categorically refused, never falling back to an embedded data URI	F-016	Must	BR-17
PR-043	An image's broken/ok status shall only be set after an explicit load probe	F-016	Should	BR-07
PR-044	The system shall generate an image from a text prompt via a user-supplied API key, with no backend proxy	F-017	Must	no BRD link
PR-045	A generated image's bytes shall be stored only in this browser (IndexedDB); the design shall carry only an id	F-017	Must	no BRD link
PR-046	An AI generation request shall never be automatically retried	F-017	Must	BR-06
PR-047	The system shall generate a structured email draft from a brief, targeting an existing preset's real fields	F-018	Must	no BRD link

PR id	Requirement	Feature	Priority	BR link
PR-048	The set of AI-fillable fields shall be derived from the preset's actual block tree, not hand-maintained per preset	F-018	Must	BR-26
PR-049	A user-supplied CTA shall remove the corresponding field(s) from the AI request entirely	F-018	Should	BR-27
PR-050	An AI brief response shall be validated as a whole; a shape/limit violation shall reject the entire response	F-018	Must	BR-28
PR-051	A brief-generated email shall carry a human-clearable "unreviewed" indicator until explicitly marked reviewed	F-018	Should	no BRD link
PR-052	The system shall allow independent model/provider selection per AI capability (text vs. image)	F-019	Should	no BRD link
PR-053	A per-generation quality/temperature choice shall override the task's configured default for that one call only	F-019	Should	no BRD link
PR-054	The system shall support an ordered, per-task fallback list of alternate models/providers	F-020	Could	BR-29
PR-055	A fallback shall be attempted only for network/timeout/rate-limit failures, never for bad-key/content-policy/malformed-response failures	F-020	Must	BR-29

PR id	Requirement	Feature	Priority	BR link
PR-056	The system shall store at most one API key per provider, defaulting to in-memory-only	F-021	Must	BR-08
PR-057	A replacement key shall only be persisted after passing a connection test	F-021	Must	BR-30
PR-058	Provider-echoed error text shall be redacted of key-shaped substrings before display	F-021	Must	BR-08
PR-059	The system shall provide one consolidated settings area covering profile, AI, brands, email defaults, checks, and data	F-022	Should	no BRD link
PR-060	A local display name shall be used only for local attribution, never as an authentication credential	F-022	Must	BR-10
PR-061	"Email defaults" changes shall apply only to brand-new emails, never retroactively	F-022	Must	BR-14
PR-062	The system shall support exporting and importing a single email's design as JSON	F-023	Should	no BRD link
PR-063	The system shall support a one-file backup covering brands, templates, saved blocks, images, and versions	F-023	Should	no BRD link
PR-064	A backup/export shall never include the AI key store or AI image drafts	F-023	Must	BR-15

PR id	Requirement	Feature	Priority	BR link
PR-065	"Clear all local data" shall require typing an exact confirmation phrase before it is enabled	F-023	Must	BR-16
PR-066	A malformed individual record within an import shall be dropped, not cause the whole import to fail	F-023	Should	no BRD link
PR-070	A color/font field shall store either a brand token or a literal value, resolved to a literal exactly once, immediately before render/export — the renderer shall never see an unresolved token	F-006	Must	BR-01
PR-071	Article-list, Hero-card, and other repeater fields shall enforce their stated maximum item count everywhere they are read (collectors, renderers, AI slot derivation, plain-text export)	F-003, F-004, F-018	Must	BR-18
PR-067 (Won't, documented for completeness)	Sending the exported email from within the app	—	Won't	no BRD link
PR-068 (Won't, documented for completeness)	Hosting AI-generated or uploaded images	—	Won't	no BRD link
PR-069 (Won't, documented for completeness)	Multi-user accounts, roles, or cross-device sync	—	Won't	BR-09, BR-10

9. Product-Level Non-Functional Requirements

NFR	What exists	Evidence
Email-client compatibility	Table-based layout, inline styles, Outlook conditional structure, no flex/grid/`position:`, bulletproof buttons, mobile stacking via one shared media query	`src/lib/render.ts`; cross-checked by `render.test.ts`, `presets.test.ts`
Size budget	Soft warning ≥90KB, hard error ≥102KB (UTF-8 bytes of the real exported HTML) — modeled on Gmail's clip point	`src/lib/checks.ts` (`SIZE_NEAR_LIMIT`/`SIZE_OVER_LIMIT`)
Accessibility (of the *output email*)	WCAG AA contrast (4.5:1 normal / 3:1 large text) checked in light and dark; minimum 12px text; minimum 44px tap target; required alt text on composite-block images	`src/lib/contrast.ts`, `src/lib/checks.ts`
Accessibility (of the *builder UI itself*)	**Not Found in Codebase** — no accessibility statement, ARIA audit, or keyboard-navigation test suite found for the editor application itself (as opposed to its output)	
Performance	Checks debounced 500ms; autosave debounced 2s; AI calls self-throttled to a 2s minimum gap and one-in-flight per task; a 90s client-side AI request timeout. **Not Found in Codebase:** any load-testing artifact or app-level performance budget	`src/lib/checks.ts`, `src/lib/ai/provider.ts`
Privacy	The AI API key defaults to in-memory-only; "Remember on this device" is explicitly disclosed as unencrypted storage; the key is structurally excluded from the app's state store and from backups; provider error text is redacted before display. **Not Found in Codebase:** any privacy policy document, or any telemetry that would itself be a privacy surface (none exists)	`src/lib/ai/key-store.ts`, `src/lib/ai/provider.ts`

NFR	What exists	Evidence
Offline / local storage limits	`localStorage` budget ~5MB (`STORAGE_LIMIT_BYTES`, approximate — not a real browser-reported figure); IndexedDB for AI image drafts (no stated cap, browser-dependent); the app otherwise functions fully offline except AI features, which require network access to `api.openai.com`	`src/lib/storage.ts`

10. Data From the User's Perspective

Everything lives in this one browser. There is no server, no account, and no sync. The table below is what a user should know before relying on this tool.

What	Where it lives	Approximate limit	Leaves the browser?
Email designs (`SavedTemplate[]`)	`localStorage: email-builder:templates:v1`	Shared ~5MB budget with everything below	Only via explicit JSON export (per-email) or full backup export
Brand kits (`Brand[]`)	`localStorage: email-builder:brands:v1`	Same shared budget; no per-category cap found	Only via explicit brand JSON export or full backup export
Saved blocks/rows (`SavedBlock[]`)	`localStorage: email-builder:blocks:v1`	Same shared budget	Only via explicit saved-blocks JSON export or full backup export
Image library (`LibraryImage[]`)	`localStorage: email-builder:images:v1`	Same shared budget	Only via explicit image-library JSON export or full backup export
Version history (`VersionRecord[]`)	`localStorage: email-builder:versions:v1`	20 per email; this category is pruned first under quota pressure	Only via full backup export
Dismissed Checks issues	`localStorage: email-builder:ignored:v1`	Same shared budget	Not included in backup/export
Merge-tag sample/preview values	`localStorage: email-builder:sample:v1`	Same shared budget	Never — editor-only convenience, never reaches an export or saved template
Local profile display name	`localStorage: email-builder:profile:v1`	Trivial size	Never leaves the browser; used only for local attribution labels
Org-wide new-email defaults	`localStorage: email-builder:email-defaults:v1`	Trivial size	Not included in backup/export
Per-task AI model routing config	`localStorage: email-builder:ai-task-routing:v1`	Trivial size	Not included in backup/export; contains provider id + model string only, never a key
AI text-generation consent flags	`localStorage: email-builder:ai-text-consent:v1`	Trivial size	Never — a local record of which provider you've agreed to send brief text to

What	Where it lives	Approximate limit	Leaves the browser?
AI provider API key(s)	`localStorage: email-builder:aikeys:v2` (opt-in only; in-memory by default)	One key per provider	**Sent only to that provider's own API** (e.g. `api.openai.com`) when generating; never included in any export, backup, or saved block — enforced structurally and by dedicated tests
AI-generated image bytes ("drafts")	IndexedDB, database `email-builder-drafts`	Browser-dependent, no app-enforced cap	Never — a design/export/backup carries only an id; an exported document always substitutes a placeholder URL instead of real bytes
Last-opened email pointer	`localStorage: email-builder:session:v1`	Trivial size	Never

What is never stored at all: the exported HTML/plain-text itself (produced on demand, not persisted); any record of what was sent or to whom (the app never sends); any analytics/usage data (none is collected — see Part 4 §13).

What a user should actively know: closing the browser/clearing site data without first using "Export all data" (Settings → Data & storage) **permanently loses everything** in this table except whatever was separately exported as JSON. There is no cloud copy anywhere.

Part 4 of 4 — Sections 11–16: Release Scope, Risks, Analytics, QA, Traceability, Glossary

> Continues from `PRD-03-flows-and-requirements.md`. Uses the same `F-xxx` / `PR-xxx` / `BR-xx` ids established in Parts 1–3, not renumbered.

---

11. Release Scope

There is no versioned release history beyond a single `package.json` version (`0.1.0`) and an incrementing feature log in `docs/PROGRESS.md` — **Not Found in Codebase: any tagged release, changelog with version numbers, or release-branch strategy.** "v1" below means "the product as it exists at the commit this PRD was written against" (`9abc092`, confirmed via `git log`).

v1 includes (as built — all 23 features from Part 1 §4)

Canvas & editing core; row layouts; atomic blocks; composite blocks; starter presets; brand kit & switcher; dark mode; merge tags; UTM tagging; plain-text export; HTML export & preview; Checks quality engine; saved blocks library; auto-save; version history; image library; AI image generation; AI brief generation; per-task AI model routing; provider fallback chains; AI key management; Settings hub & profile; import/export & backup.

Known gaps and limitations (Fact — each explicitly acknowledged in the codebase/docs, not inferred)

Gap	Why it exists	Evidence
No backend	By design — client-only SPA	`vite.config.ts` has no `server.proxy` / `server` key
No image hosting	Deliberately out of scope so far	`imageHosting.configured === false`
No test-send	No ESP integration of any kind exists	README: "No sending"
No multi-user / cross-device sync	No backend, no accounts	README "Known limitations"
No AI usage cap or cost estimator	Not built	Confirmed by exhaustive grep across `src/lib/ai/*`
Only one AI provider registered	A second provider slot exists in the data model but is unfilled	`src/lib/ai/openai.ts`, `PROVIDERS` array
Checks don't verify link liveness	Classification only, not a fetch	`checks.ts` comment: "Links are classified, not fetched"
No CI pipeline evidence found	Not confirmed either way within this review's scope	No CI config file found in the repository

Backlog candidates (Inference — priority and rationale are this PRD's own judgment; items themselves are drawn from `README.md`'s own "Ideas for next versions" plus gaps above, not invented)

Candidate	Priority (Inference)	Rationale
Image hosting (so generated/uploaded images can ship as real bytes)	High	Directly closes the single biggest "doesn't fully work yet" gap in F-016/F-017
Supabase persistence (auth + a `templates` table, design JSON in `jsonb`)	High	Named explicitly in the README; the only path to real multi-user/cross-device use without a from-scratch backend
Test-email sending	Medium	Closes the loop between "export" and "verify it renders," without leaving the tool
A second real AI provider	Medium	Makes the already-built fallback chain (F-020) meaningfully useful, not just same-provider-different-model
AI usage/cost visibility	Medium	Directly addresses the "no cost control" risk (§12)
Merge-tag picker with groups/display conditions	Medium	Scales personalization sophistication beyond the current 6 built-in tags
AI "generate section" (targeted regeneration of one block, not a whole-email brief)	Medium	A smaller, faster-feedback alternative to full brief regeneration
Countdown-timer block	Low	A common promotional pattern not currently supported
Template-gallery thumbnails	Low	Browsing improvement for the Templates modal

12. Risks and Open Questions

Risks

Risk	Category	Note
Total data loss on a lost/corrupted browser profile	Data	No automatic off-device backup exists; only a manual "Export all data" action
`localStorage` quota exhaustion	Operational	Only version history is auto-pruned; templates/brands/images/saved blocks are unbounded with no automated relief
Unreviewed AI content shipping in a real send	Process	`AI_DRAFT_UNREVIEWED` is a warning, never a block (BR-05 applies even to AI output)
AI hallucination in brief-generated copy	Process	The system prompt forbids inventing facts and requires a bracket placeholder instead, but compliance is not mechanically enforced beyond the strict-schema shape
AI model deprecation breaking a feature silently	Integration	Already happened once in this project's own history (`gpt-image-1` stopped existing) — model lists are hardcoded and need a manual update
Direct, uncapped AI billing exposure per user	Operational	No usage cap or cost estimator exists anywhere in the app
Single point of failure: one browser, one person	Business continuity	If that person/browser is unavailable, the team's brand kits/templates are inaccessible without a prior backup export

Open questions (Inference — these are genuinely unresolved in the codebase, not rhetorical)

#	Question
Q1	Should the unreviewed-AI-content warning (`AI_DRAFT_UNREVIEWED`) ever become a hard block before export, or stay purely advisory forever, consistent with the product's "never block" philosophy?
Q2	Is image hosting intended to be built in-house (e.g., a small upload API) or delegated to a third-party CDN the user configures themselves?

#	Question
Q3	Is multi-user/cross-device support actually on a roadmap, or are the `createdBy`/`canEditBrand` seams permanent unused scaffolding?
Q4	Should AI cost visibility be a per-call estimate, a running session counter, or left entirely to the user's own OpenAI dashboard?
Q5	Is a second AI provider planned, or is the fallback-chain feature (F-020) intentionally future-proofing for a scenario with no current timeline?

---

13. Analytics and Telemetry

Not Found in Codebase. Exhaustive — no event tracking, no error-reporting SDK (e.g. Sentry), no usage-metrics collection, no A/B testing framework, and no first-party or third-party analytics script anywhere in `src/` or `index.html`. Confirmed by grep across the full source tree for common analytics identifiers (`analytics`, `gtag`, `segment`, `mixpanel`, `amplitude`, `sentry`, `posthog`) — no hits outside this document and the BRD's own prose discussing their absence.

This means: the product has no way to know, from within itself, how many emails are created, how often AI features are used, which Checks rules fire most often in practice, or whether any user has ever completed a given flow. Every "metric" discussed in §2 is a point-in-time UI readout, not telemetry.

---

14. QA and Acceptance — Manual Verification Checklist

A practical, per-feature-area manual regression checklist, derived directly from the acceptance criteria in Part 2 §5. Intended for a human tester running through the app; it does not replace (and in fact is already covered by) the automated suite in §15.

- [ ] **Canvas/editing (F-001, F-002):** Add, drag-reorder, duplicate, and delete a block and a row; confirm Ctrl+Z/Ctrl+Shift+Z undo/redo across a coalesced text edit and a structural change.
- [ ] **Blocks (F-003, F-004):** Add one of each of the 9 atomic + 4 composite block types; confirm a Hero layout change pads/trims cards correctly; confirm hiding the Footer removes it from Preview but not from the dimmed canvas.
- [ ] **Presets (F-005):** Apply two different presets back-to-back; confirm each is a fresh, non-aliased copy (edit one, confirm the preset picker's thumbnail for the same preset is unaffected).
- [ ] **Brand kit (F-006):** Create a brand, set a manual hex override on one field, switch to another brand, confirm the override survives while token-sourced fields re-theme.
- [ ] **Dark mode (F-007):** Toggle dark mode off, export, and diff against a dark-mode-on export of the identical design — confirm the off-export has no `dm-` classes or meta tags.
- [ ] **Merge tags (F-008):** Insert `{{first_name}}`, switch format to Mailchimp, confirm the export shows `*|FNAME|*` and the stored design is unchanged.
- [ ] **UTM (F-009):** Enable UTM, add a link that already has a `utm_source`, confirm it is not duplicated; confirm the unsubscribe link is skipped but counted.
- [ ] **Plain text (F-010):** Export plain text for an email containing a long URL; confirm it is not split mid-URL.
- [ ] **Export/Preview (F-011):** With at least one Checks error present, confirm Copy/Download remain enabled in the Export dialog.
- [ ] **Checks (F-012):** Ignore an issue, reload the app, reopen the same email, confirm it is still ignored; press "Check images load" and confirm only probed images can show as broken.
- [ ] **Saved blocks (F-013):** Save a block containing a brand-token color, insert it under a different brand, confirm it re-themes.
- [ ] **Auto-save (F-014):** Make an edit on a brand-new email, wait ~2s, confirm a record appears under "Drafts (auto-saved)" in Templates.
- [ ] **Version history (F-015):** Restore an older version, then confirm the pre-restore state is itself available to restore back to.
- [ ] **Image library (F-016):** Attempt "Upload from device" on an Image block and confirm the explicit refusal message; add a valid https URL and confirm it probes successfully.
- [ ] **AI image (F-017):** With no key configured, confirm the AI image panel opens directly into Settings; after generating, confirm "Add to email" is disabled until alt text is entered.
- [ ] **Brief generation (F-018):** Submit a brief with a CTA label filled in; confirm the created email uses that exact label, not an AI-authored one; confirm the "unreviewed" Checks warning appears and clears on "Mark as reviewed."
- [ ] **Model routing & fallback (F-019, F-020):** Configure a fallback model for one task; simulate/observe a rate-limit failure and confirm the fallback is attempted (vs. an invalid-key failure, which should not trigger a fallback).

- [ ] **AI key management (F-021):** Enter a deliberately invalid replacement key, confirm "Test and save" fails and the previously working key still shows as Connected.
- [ ] **Settings hub & profile (F-022):** Set a display name, create a new brand, confirm the brand's "created by" reflects that name; change an "Email defaults" setting and confirm an already-open email is unaffected.
- [ ] **Import/export/backup (F-023):** Export a full backup, then "Clear all local data" (confirming the typed-`DELETE` gate), then import the backup and confirm brands/templates/saved blocks/images/versions are all restored while the AI key remains unset (never backed up).

---

15. Traceability Matrix

BR id (from `docs/brd/BRD-02.md` §13) → PR id (Part 3 §8) → Feature → primary Screen → primary automated test file(s), confirmed to exist and pass in the test run at the top of Part 1.

BR id	PR id	Feature	Screen	Test file(s)
BR-01	PR-070	F-006	BrandsModal, TopBar (switcher)	`src/test/brand.test.ts`
BR-02	PR-014	F-006	TopBar (switcher)	`src/test/brand.test.ts`
BR-03	PR-015	F-006	BrandsModal	`src/test/brand.test.ts`
BR-04	PR-016	F-006	BrandsModal	`src/test/brand.test.ts`
BR-05	PR-028	F-011, F-012	ExportHtmlModal, ChecksPanel	`src/test/checks.test.ts`, `src/test/app.test.tsx`
BR-06	PR-046	F-017	AiImagePanel	`src/test/ai.test.ts`, `src/test/ai-ui.test.tsx`
BR-07	PR-031, PR-043	F-012, F-016	ChecksPanel, ImageLibrary	`src/test/checks.test.ts`, `src/test/image-audit.test.ts`, `src/test/image-library.test.ts`
BR-08	PR-056, PR-058	F-021	AI settings view	`src/test/ai.test.ts`, `src/test/ai-providers.test.tsx`
BR-09	(no single Must PR — governs F-006/F-013/F-016/F-023 jointly)	F-006, F-013, F-016, F-023	—	`src/test/backup.test.ts`, `src/test/storage.test.ts`
BR-10	PR-060	F-022	Settings → Profile	`src/test/user.test.ts`
BR-11	PR-035	F-014	TopBar (save status)	`src/test/storage.test.ts`, `src/test/app.test.tsx`
BR-12	PR-039, PR-040	F-015	HistoryModal	`src/test/versions.test.ts`

BR id	PR id	Feature	Screen	Test file(s)
BR-13	PR-037, PR-038	F-014, F-015	TopBar, HistoryModal	`src/test/storage.test.ts`, `src/test/versions.test.ts`
BR-14	PR-061	F-022	Settings → Email defaults	`src/test/email-defaults.test.ts`
BR-15	PR-064	F-023	Settings → Data & storage	`src/test/backup.test.ts`
BR-16	PR-065	F-023	Settings → Data & storage	`src/test/backup.test.ts`
BR-17	PR-042	F-016	ImageLibrary	`src/test/image-library.test.ts`
BR-18	PR-071	F-003, F-004, F-018	RightPanel (ArticleListEditor/HeroEditor)	`src/test/blocks.test.ts`, `src/test/brief-slots.test.ts`
BR-19	PR-023	F-009	RightPanel (Body/Email settings)	`src/test/utm.test.ts`
BR-20	PR-024	F-009	ExportHtmlModal	`src/test/utm.test.ts`
BR-21	PR-020	F-008	RightPanel (Body/Email settings)	`src/test/merge.test.ts`
BR-22	PR-018	F-007	BrandsModal, PreviewModal	`src/test/dark.test.ts`
BR-23	PR-019	F-007	BrandsModal	`src/test/dark.test.ts`
BR-24	PR-010	F-005	NewEmailModal	`src/test/presets.test.ts`
BR-25	PR-011, PR-034	F-005, F-013	NewEmailModal, SaveBlockDialog	`src/test/presets.test.ts`, `src/test/saved-blocks.test.ts`
BR-26	PR-048	F-018	BriefForm, BriefReview	`src/test/brief-slots.test.ts`
BR-27	PR-049	F-018	BriefForm	`src/test/brief-generate.test.ts`, `src/test/brief-orchestrate.test.ts`

BR id	PR id	Feature	Screen	Test file(s)
BR-28	PR-050	F-018	BriefReview	`src/test/brief-validate.test.ts`
BR-29	PR-055	F-020	Settings → AI	`src/test/ai-run-task.test.ts`
BR-30	PR-057	F-021	AI settings view	`src/test/ai-providers.test.tsx`

Features with no BRD-cited business rule (F-001, F-002, F-003 beyond BR-18, F-004 beyond BR-18, F-010, F-011 beyond BR-05, F-019): these are real, tested, shipped capabilities that the BRD simply did not formalize as a numbered business *rule* (a constraint whose violation would break a guarantee) — they are covered by their own test files (`render.test.ts`, `plaintext.test.ts`, `dnd.test.ts`, `rte.test.tsx`, `ai-tasks.test.ts`, `ai-task-settings.test.ts`, `ai-task-ui.test.tsx`) and by their PR rows in Part 3 §8, just with "no BRD link."

---

16. Glossary

Term	Meaning
F-xxx	A feature id from the inventory in Part 1 §4, stable across all 4 PRD parts
PR-xxx	A functional requirement id from Part 3 §8, stable across all 4 PRD parts
BR-xx	A business-rule id from `docs/brd/BRD-02.md` §13
MoSCoW	Must/Should/Could/Won't prioritization scheme used in Part 3 §8 (Inference-tagged, since no prioritization document exists in the codebase)
Slot	One AI-fillable text field in the brief-generation feature (F-018), derived automatically by walking a preset's real block tree
Task (AI)	One of the product's two AI capabilities — `text.brief` or `image.generate` — independently routed to a provider/model (F-019)
Draft (AI image)	A locally generated image's bytes, held in this browser's IndexedDB; the design only ever stores its id
Resolve at the boundary	The rule that brand color/font tokens become literal values exactly once, immediately before render/export (BR-01)
Checks	The non-blocking email-quality engine (F-012); an "issue" is one finding with a severity, message, and stable id
Preset	One of 8 fixed, brand-neutral starter structures (F-005)
Saved block	A frozen, reusable copy of a block or row (F-013)
Auto-saved draft	A `SavedTemplate` record not yet given an explicit name
Fallback chain	An ordered, user-configured list of alternate AI provider/model choices (F-020)
BYOK	Bring-your-own-key — the user supplies their own OpenAI API key, billed directly to them, no backend proxy
Profile (display name)	A local, unauthenticated label used only for `createdBy` / `updatedBy` attribution (F-022) — not a login
Backup	A one-file JSON export of brands, templates, saved blocks, images, and versions, deliberately excluding AI keys and drafts (F-023)

Appendix A: Verification Notes

Every **Inference** and **Not Found in Codebase** statement across `PRD-01-overview.md`–`PRD-04-release-and-appendix.md`, by section, with the reasoning behind each tag. Anything not listed here is a **Fact** — stated directly in source code, tests, `docs/PROGRESS.md`, or `docs/brd/`. Real counts in this PRD (test suite, type check) were run fresh at write time, not quoted from memory or from `docs/PROGRESS.md`'s own log — see the verification block at the top of `PRD-01-overview.md`.

Part 1 (PRD-01-overview.md)

§1 Product Overview

- **Not Found in Codebase** — a written vision statement, mission statement, or product-strategy document. *Reasoning:* none exists; the "Vision" line is an **Inference**, synthesized from `README.md`'s and `docs/PROGRESS.md`'s own framing of each feature's purpose, presented as a quote of this document's own synthesis (not a quote found verbatim anywhere).
- **Inference** — "Positioning" (the ESP-editor/Canva-alternative comparison). *Reasoning:* no competitive-positioning document exists; this is a reasonable, clearly-labeled analyst judgment based on the product's actual architecture (client-only, BYOK AI, built-in quality engine), not a stated claim.
- **Inference** — "Not a general page-builder." *Reasoning:* no explicit statement rules this out; drawn from every rendering constraint (table layout, no flex/grid, inline styles) being specifically email-oriented.

§2 Goals, Non-Goals, Success Metrics

- **Inference** — All 7 goals (G1–G7). *Reasoning:* no OKR/goals document exists; each is derived from the stated purpose of the feature(s) that implement it, per `docs/PROGRESS.md`'s own framing.
- **Fact, with Inference labeling kept visible for honesty** — the 7 non-goals are each backed by a direct code/doc citation (README, `imageHosting.configured`, `user.ts` comment, exhaustive grep results, `checks.ts` comment, absence of i18n framework) — these are Facts, not inferences, despite living in a "goals" section; flagged here only so a reviewer doesn't need to re-derive that distinction.
- **Not Found in Codebase** — Any analytics, telemetry, or usage-metrics instrumentation. *Reasoning:* exhaustive, re-confirmed independently for this PRD (see Part 4 §13's separate grep pass).
- **Inference** — The entire "observable in-app counter" framing as a success-metrics substitute. *Reasoning:* these are real UI elements (Fact), but calling them "success metrics" is this document's own interpretive framing, since the product itself never calls them that and never aggregates them.

§3 Personas and Use Cases

- **Not Found in Codebase** — A brand-owner-only, approver, viewer-only, or admin role. *Reasoning:* `canEditBrand()` / `getAccessibleBrands()` are documented in-code as future seams that currently grant

unconditional access; no second role is ever checked against.
- **Inference** — The persona label "Marketing/Communications editor" itself. *Reasoning:* the code never names a role; "Vee MARCOM team" (Fact, from `docs/PROGRESS.md`) is generalized into a persona label for this document's structure.

§4 Feature Inventory

- **Inference** — The specific grouping of 23 features (F-001–F-023), their Area labels, and the Status column's Complete/Partial judgment for F-016/F-017. *Reasoning:* no feature-list document exists; the grouping/status judgment is this PRD's own organization of Facts already established (e.g., "URL-only by design" for F-016 is a direct quote/fact from the code comments, but calling that "Partial" rather than "Complete" is this document's classification choice).

Part 2 (PRD-02-feature-specs.md)

General

- **Inference** — Every Given/When/Then acceptance-criteria block, across all 23 feature specs. *Reasoning:* no formal acceptance-criteria or user-story document exists in the repository; every block is phrased from tested/documented behavior and cross-checked against source, not copied from an existing artifact.
- **Inference** — Every "User story" line (the "As the editor, I want... so that..." framing). *Reasoning:* same basis — these are this document's restatement of a real capability in user-story form, not a quote of an existing user story (none exist in the repo).

F-018 (Generate email from a brief)

- **Not Found in Codebase** — "5B," "5C," and a standalone "4A" feature. *Reasoning:* `docs/PROGRESS.md` itself explicitly states these do not exist as separate features (only 5A plus its 4 amendments); this PRD repeats that finding rather than inventing or silently dropping the reference.

Part 3 (PRD-03-flows-and-requirements.md)

§8 Functional Requirements

- **Inference** — Every MoSCoW priority (Must/Should/Could/Won't) across all 71 PR ids. *Reasoning:* no prioritization document exists; the rule used (Must = breaks a BR-cited guarantee or a core authoring workflow if removed; Should = documented deliberate behavior, not load-bearing for correctness; Could = UX nicety; Won't = explicitly out of scope) is stated plainly in the section so a reviewer can disagree with a specific call without needing to guess the method.

- **Inference** — The PR-001...PR-071 wording itself (requirement phrasing). *Reasoning:* no requirements-document predates this PRD; each requirement is this document's restatement of tested/documented behavior as a "shall" statement.

§9 Non-Functional Requirements

- **Not Found in Codebase** — Accessibility engineering for the *builder UI itself* (as opposed to its output email, which is extensively engineered and a Fact). *Reasoning:* no ARIA audit, accessibility statement, or keyboard-navigation test suite targeting the editor application was found.
- **Not Found in Codebase** — Any load-testing artifact or app-level performance budget (distinct from the email-size Checks rules, which govern *output* quality, not app performance). *Reasoning:* none found in `src/` or `docs/`.
- **Not Found in Codebase** — Any privacy-policy document. *Reasoning:* none exists; the privacy-relevant *engineering* (key handling, redaction) is a Fact and is cited separately from this specific absence.

§10 Data From the User's Perspective

- No new Inference/Not-Found tags — this section restates Facts (storage key names, what leaves the browser) already established in `docs/brd/BRD-02.md` §15–16 and re-verified against source for this PRD.

Part 4 (PRD-04-release-and-appendix.md)

§11 Release Scope

- **Not Found in Codebase** — Any tagged release, changelog with version numbers, or release-branch strategy. *Reasoning:* only a single `package.json` version (`0.1.0`) and an incrementing prose feature log (`docs/PROGRESS.md`) exist; "v1" in this PRD is this document's own label for "the product at the commit this PRD was written against," not a label the product uses itself.
- **Inference** — Every backlog-candidate priority and rationale. *Reasoning:* the candidates themselves are drawn directly from `README.md`'s own "Ideas for next versions" list (a Fact) plus the gaps already established elsewhere in this PRD; the priority ranking and the one-line rationale per item are this document's own judgment, explicitly labeled as such in the section heading.

§12 Risks and Open Questions

- **Inference** — All 5 open questions. *Reasoning:* by definition genuinely unresolved in the codebase (that is the point of listing them) — each is grounded in a real, cited ambiguity or seam (e.g., Q3 cites the real `createdBy`/`canEditBrand` seams) rather than a speculative question with no basis.
- The risks table itself largely restates Facts already established and cited elsewhere in this PRD and in the BRD; no new Inference/Not-Found tags beyond the open questions.

§13 Analytics and Telemetry

- **Not Found in Codebase.** *Reasoning:* exhaustive grep across the full source tree for common analytics identifiers (`analytics`, `gtag`, `segment`, `mixpanel`, `amplitude`, `sentry`, `posthog`), re-run independently for this PRD rather than solely relying on the equivalent finding already made for the BRD — same result, now double-confirmed.

§14 QA and Acceptance Checklist

- **Inference** — The checklist itself (phrasing, grouping, and the specific steps chosen). *Reasoning:* no QA/test-plan document exists in the repository; every line is derived directly from an acceptance-criteria block already established in Part 2 §5, restated as a manual-tester action rather than an automated assertion.

§15 Traceability Matrix

- **Fact** — Every BR/PR/Feature/Screen mapping restates ids and files already established earlier in this PRD and in the BRD.
- **Note, not a tag:** BR-09 has no single "Must"-priority PR directly citing it (it is cited as "no single Must PR — governs jointly" across F-006/F-013/F-016/F-023) because it describes a cross-cutting architectural absence (no sync mechanism) rather than one feature's behavior — this is accurately represented as a joint governance relationship rather than forced into one artificial PR row.
- **Fact** — Every test file name in this matrix was confirmed to exist via `ls src/test/*.test.*` and to be part of the passing 41-file / 950-test run reported at the top of Part 1; none were quoted from memory.

§16 Glossary

- No new Inference/Not-Found tags — restates terms already defined and sourced earlier in this PRD and in the BRD's own glossary.

Summary count

Tag	Approximate count across all 4 parts
Inference	~20 distinct statements/sections
Not Found in Codebase	~12 distinct statements/sections

Consistent with the BRD's own finding: the concentrations are in sections a product-management template assumes exist for any product (a written **vision statement**, a **goals/OKR document**, **analytics/telemetry**, a **prioritization document**, a **release/changelog process**) — none of which this single-developer-history, no-backend, no-analytics internal tool actually has. That mismatch, not a gap in this research, is what drives nearly every tag in this file.